

АРХИТЕКТУРА ПРЯМОГО ВЛАДЕНИЯ ДАННЫМИ · ТЕХНИЧЕСКАЯ СПЕЦИФИКАЦИЯ

Платформа «Турбаза» — Полный технический документ

Архитектурные принципы, трехуровневая топология, криптографический контур и экономика API

Технический документ платформы «Турбаза»

Концептуальные основы и мотивация платформы «Турбаза»

Основными движущими силами создания платформы «Турбаза» являются обеспечение цифрового суверенитета государства, внедрение принципов справедливого владения данными и развитие независимой экосистемы разработки программного обеспечения.

- Цифровой суверенитет: Архитектура платформы позволяет государству сохранять контроль над данными своих граждан в пределах юрисдикции, обеспечивая при этом возможность контролируемого межгосударственного обмена информацией при соблюдении высочайших стандартов безопасности и государственного управления.
- Справедливое владение данными: В основе платформы лежит фундаментальный принцип: каждый человек и каждая семья должны обладать полным правом собственности на свою частную информацию и единолично контролировать доступ к ней, в то время как общественно значимые данные должны оставаться в распоряжении общества, исключая их монополизацию частными корпорациями.
- Независимая разработка ПО: Платформа создает открытую экосистему, в которой частные лица и представители малого бизнеса могут равноправно участвовать в создании и интеграции кооперативных приложений, обеспечивая технологическую независимость и инновационное развитие.

Часть I: Базовая инфраструктура

1. Концептуальное видение и архитектура прямого владения данными

1.1. Миссия

Обеспечение технологического суверенитета через создание защищенного цифрового контура. Платформа возвращает контроль над данными и их монетизацией обратно в национальное поле, предотвращая утечку информации в коммерческий сектор и исключая несанкционированное использование данных ведомствами, посторонними определенным данным. Государство выступает как гарант прозрачности и безопасности, при этом дизайн системы гарантирует, что даже внутри государственной сети данные строго изолированы. Ключевая цель — дать независимым разработчикам возможность создавать узкоспециализированные, «лучшие в своем классе» (best-in-breed) приложения, работающие поверх отечественной инфраструктуры прямого владения данными.

1.2. Основные компоненты системы

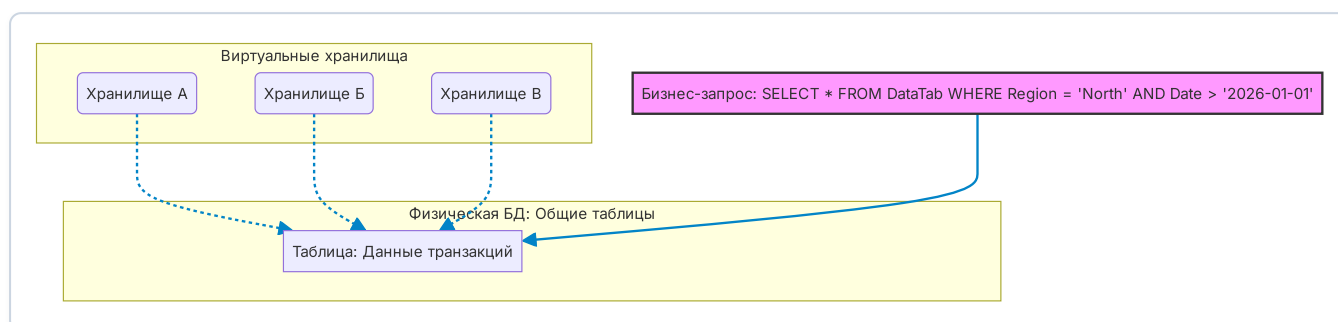
Лист (Leaf): Базовая инфраструктура и среда исполнения (runtime), обеспечивающая выполнение приложений пользователя. Это полноценная платформа исполнения (runtime environment) для модульных приложений, работающих с кооперативными схемами в локальной базе данных. «Лист» может работать в браузере на ПК или телефоне пользователя или быть встроен в «Ветку» как пункт доступа в другую Ветку.

Ветвь (Branch): Инфраструктурный шлюз районного, регионального или ведомственного уровня. Обеспечивает хостинг государственных данных, маршрутизацию зашифрованных логов и проверку прав доступа.

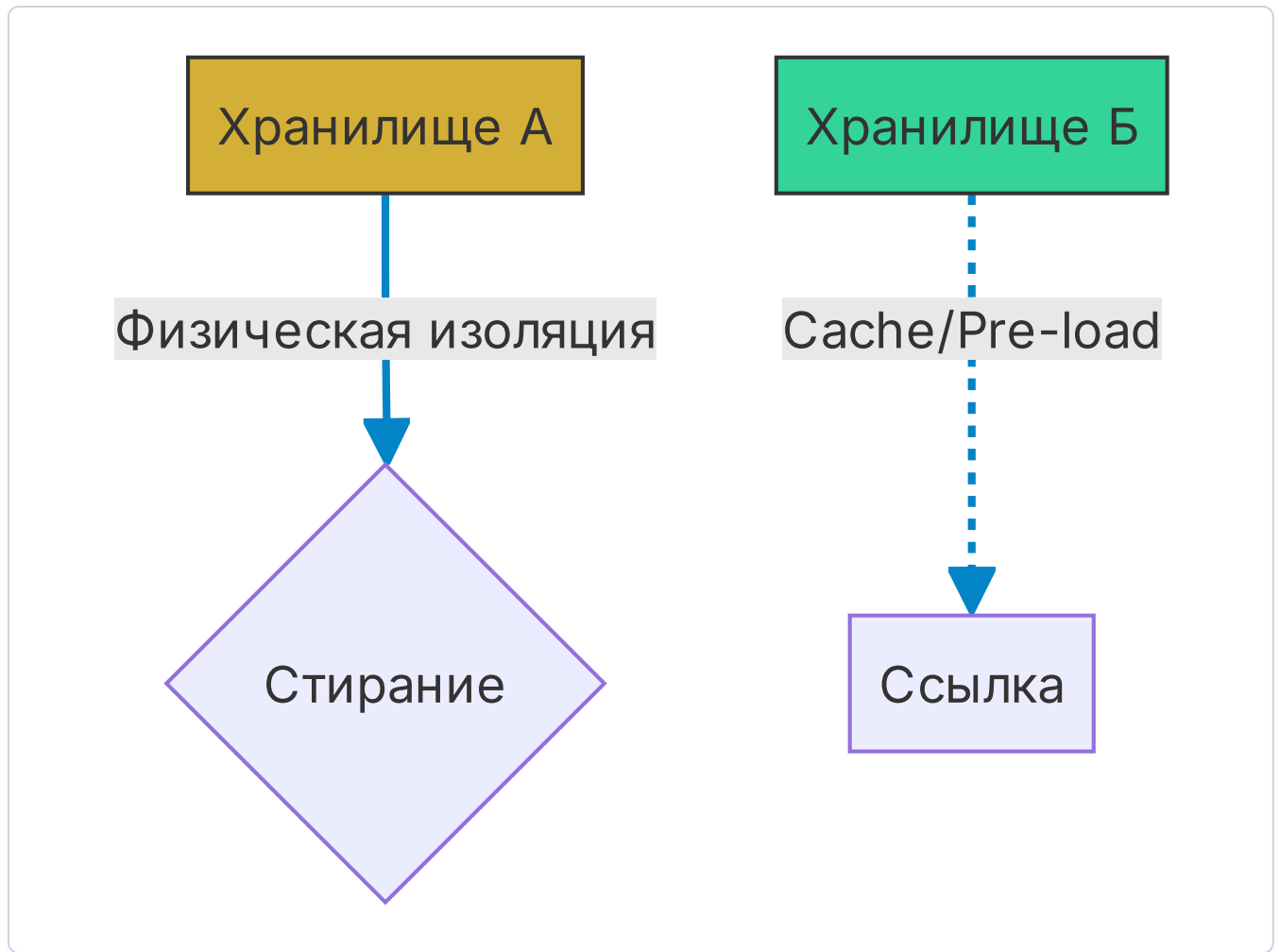
Хранилища: Изолированные сегменты данных (Государственные, Частные, Коммерческие), защищенные уникальными ключами.

1.3. Хранилища и хранение

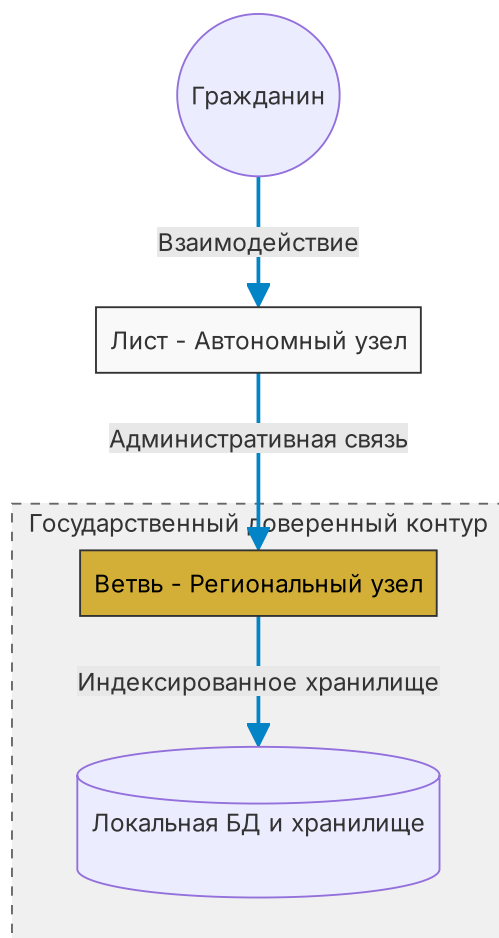
Система базируется на множестве независимых блокчейнов-хранилищ, обеспечивающих физическую изоляцию данных. В этой архитектуре используется одна физическая реляционная база данных для хранения и управления несколькими "виртуальными" хранилищами данных. Эта схема позволяет изолировать данные разных приложений или отделов друг от друга (что обеспечивает их автономное обслуживание), но при этом оставляет возможность выполнять совместные SQL-запросы к данным из разных хранилищ без необходимости в сложных механизмах интеграции или копирования данных.



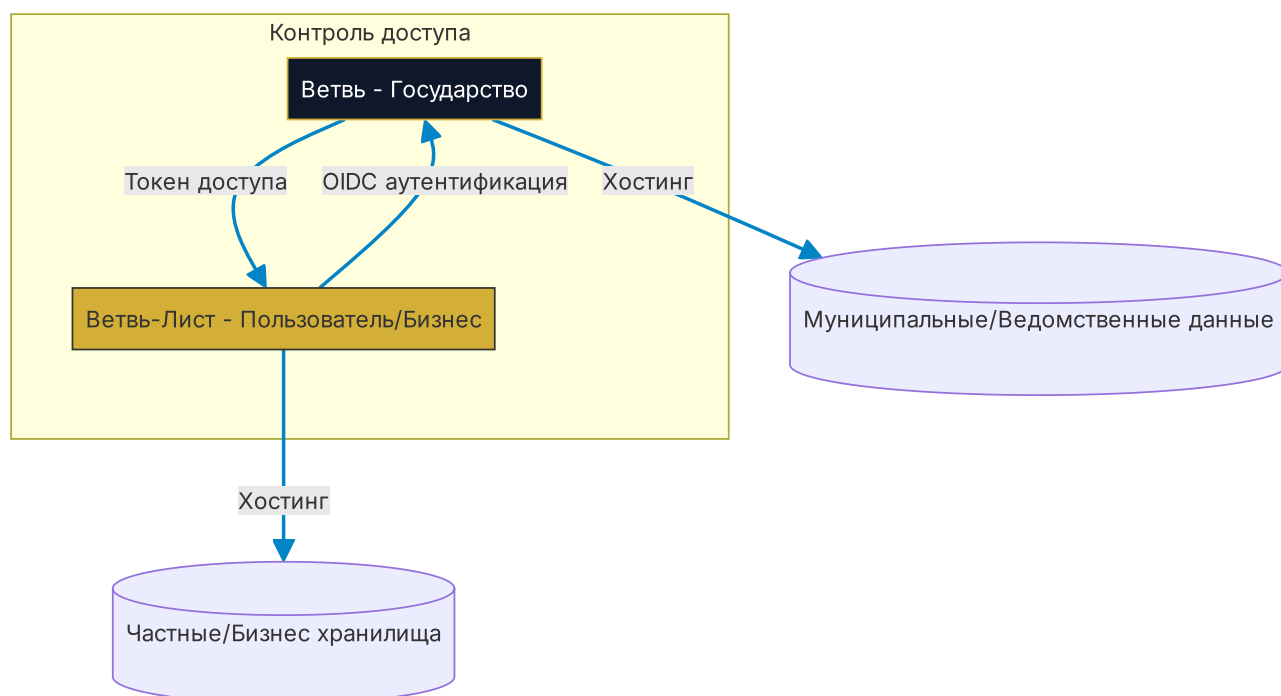
Стирание одного хранилища не влияет на другие поскольку каждый из них это отдельный блокчейн. Хранилища могут ссылаться на друг друга и могут кэшировать ссылки для скорости вывода информации и навигации дополнительных ссылок.



Государственные хранилища: Хранятся на узлах соответствующих «Ветвей». Ветвь служит «хостом» для локальных, муниципальных и региональных данных.



Частные хранилища: Пользователи/группы/бизнес могут разворачивать собственные «Ветви» в входящие в общую платформу как «Листы». Пользователи могут входить в частные и бизнес «ветки» на прямую но для получения доступа к государственным хранилищам связанными с ними или бизнесом им надо пройти через аутентификацию с государственной веткой.



Приватность: Данные в частных хранилищах полностью зашифрованы на стороне владельца.

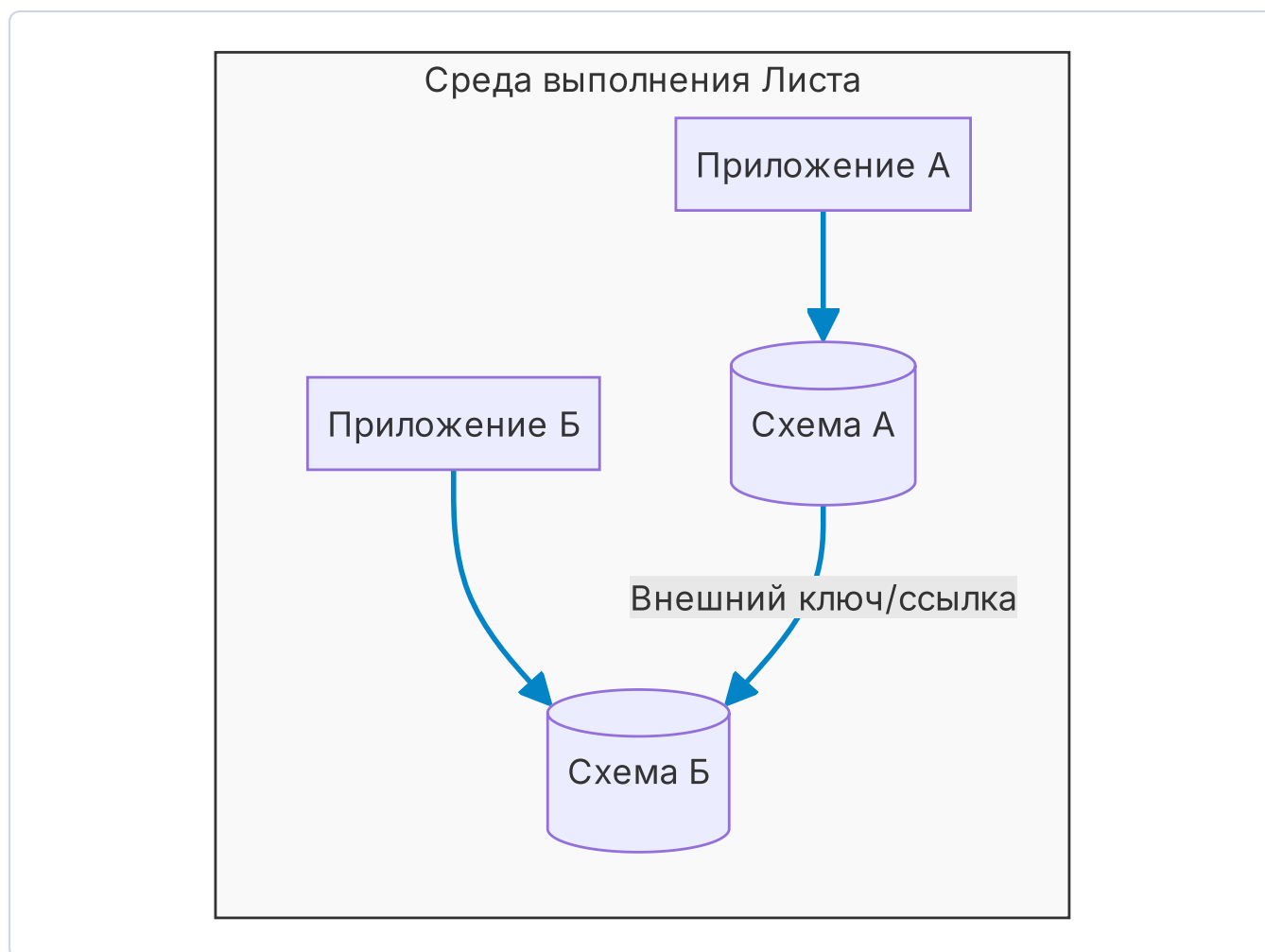
1.4. Принципы кооперативной интероперабельности

Система спроектирована для поддержки независимых разработчиков, создающих «лучшие в своем классе» (best-in-breed) приложения построена и на принципах открытой интероперабельности:

Лист (Leaf): Базовая инфраструктура, обеспечивающая выполнение приложений.

Кооперативные хранилища: Реляционные базы данных, где схемы приложений могут ссылаться друг на друга.

Принцип «Читай отовсюду, пиши только своё»: Приложения имеют право записи только в свои схемы, но могут читать данные из любых других схем (при наличии разрешений владельца), что создает синергию без дублирования.

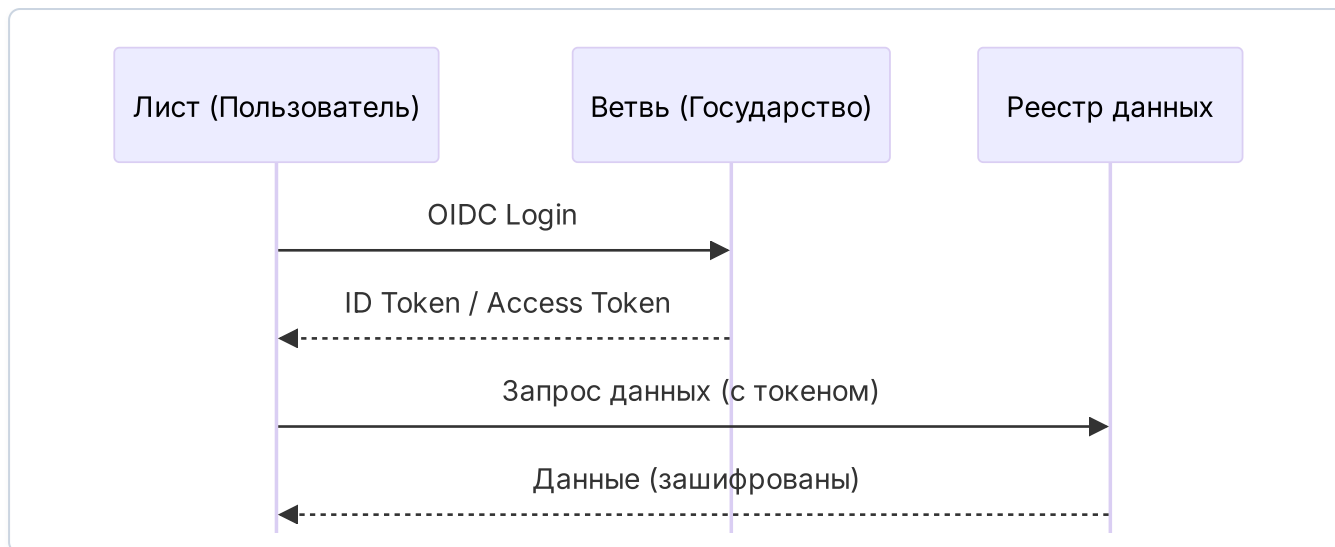


1.5. Протокол аутентификации OIDC

Для доступа к государственным данным или для взаимодействия между частными хранилищами и государственными реестрами используется протокол OpenID Connect (OIDC).

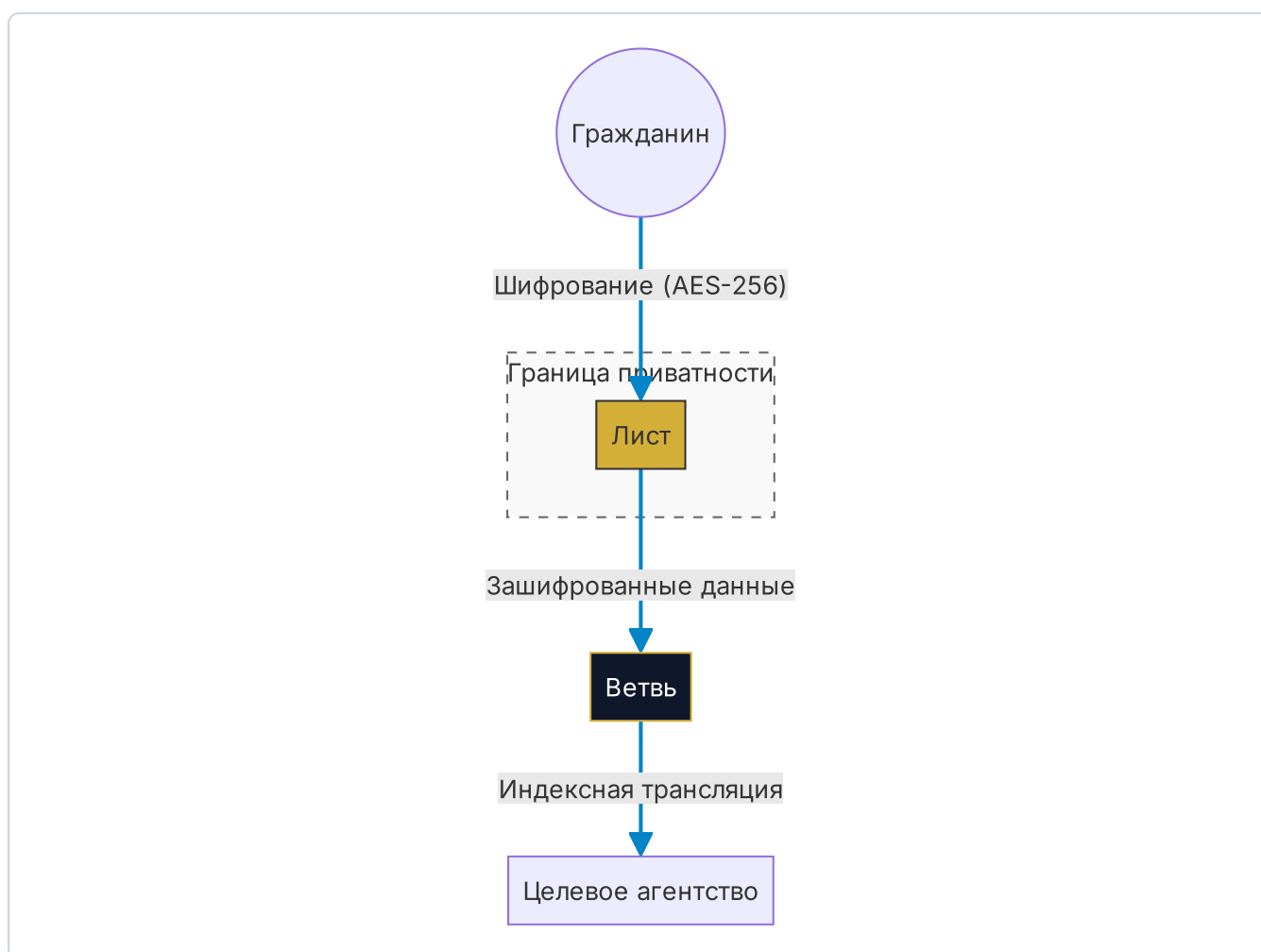
Процесс: Пользователь аутентифицируется через свою локальную «Ветвь».

Результат: Получение защищенного токена, который позволяет «Листу» пользователя безопасно запрашивать данные, разрешенные к публикации/обмену.



1.6. Протокол безопасности и Zero-Knowledge

Лист обменивается с Веткой записями об изменениях в логах транзакций хранилища. Все записи подписываются частным ключём пользователя локально на листе, до отправления в ветку и гарантируя что именно этой пользователь произвел эту запись. Частные данные (включая Keyring) шифруются локально на стороне пользователя (Лист). «Ветвь» не имеет доступа к частному контенту — она выступает как защищенный шлюз для передачи зашифрованных логов транзакций, что исключает возможность пассивного чтения данных.



1.7. Регистрация пользователей и канонический идентификатор Actor

Определение и техническое назначение Actor

В модели данных платформы «Турбаза» каждый субъект создания записей идентифицируется кортежем **Actor**:

$$\text{Actor} = (\text{User}, \text{Device}, \text{App})$$

где:

- **User** — логический идентификатор пользователя в системе;
- **Device** — идентификатор физического устройства (смартфон, планшет, рабочий компьютер);
- **App** — идентификатор конкретного приложения (Web Component / WASM), выполняющего транзакцию.

Техническое назначение: Концепт **Actor** служит исключительно для детерминированного предотвращения коллизий составных первичных ключей при параллельном создании записей одним и тем же пользователем на разных физических устройствах или через разные изолированные приложения, обращающиеся к общей реляционной схеме. Идентификатор **Actor** **принципиально не используется для анонимизации** и не несет функции сокрытия личности.

Режимы регистрации на Ветке: полная анонимность, социальная анонимность, поимённая регистрация

Анонимность, приватность и юридическая значимость действий в экосистеме «Турбазы» реализуются на уровне регистрации пользователя на узле «Ветка» (Branch). Частная информация (личные журналы, семейные и групповые хранилища, приватные заметки) хранится в частных хранилищах и никогда не попадает на Ветки. Ветки индексируют только общественную информацию с помощью древовидных структур платформы: сейчас разрабатывается индексация тем и тегов, а также страниц тредов, в будущем возможны и другие виды индексации.

Различаются два вида анонимности и поимённый режим:

- **Полная анонимность (Branch ID):**
 - Никто не знает, кто этот пользователь; известен только район, в котором он находится.
 - Пользователь анонимно подключается к районной Ветке, получая случайный локальный номер (**Number**). Ветка оставляет за собой право фиксировать факт нахождения в сетевом контуре района (по локальной маршрутизации/IP) для защиты от внешнего спама, без сбора и хранения персональных данных.
- **Социальная анонимность (в том числе Group Anonymity / слепые подписи и ZKP):**
 - В общественном контуре человек не известен, но экономически известен: за такой регистрацией стоит реальный человек, поскольку Банк России выдаёт только один кошелек на человека. Отсюда следует определённое экономическое и вытекающее из него социальное доверие: если произойдёт серьёзное нарушение, личность может быть установлена в судебном порядке. Общество и контрагенты личность не видят; её знает только Банк России.
 - Групповая анонимность подтверждает принадлежность к определённой социальной или территориальной группе (житель конкретного микрорайона, родитель ученика школы, член кооператива) без раскрытия персональной идентичности. Механизм опирается на криптографические слепые подписи (Blind Signatures) и протоколы доказательства с нулевым разглашением (ZKP), а также процедуру доверенной жеребьёвки группы номеров. Ветка заверяет право на участие, но генерируемый псевдоним криптографически отвязан от реального профиля.

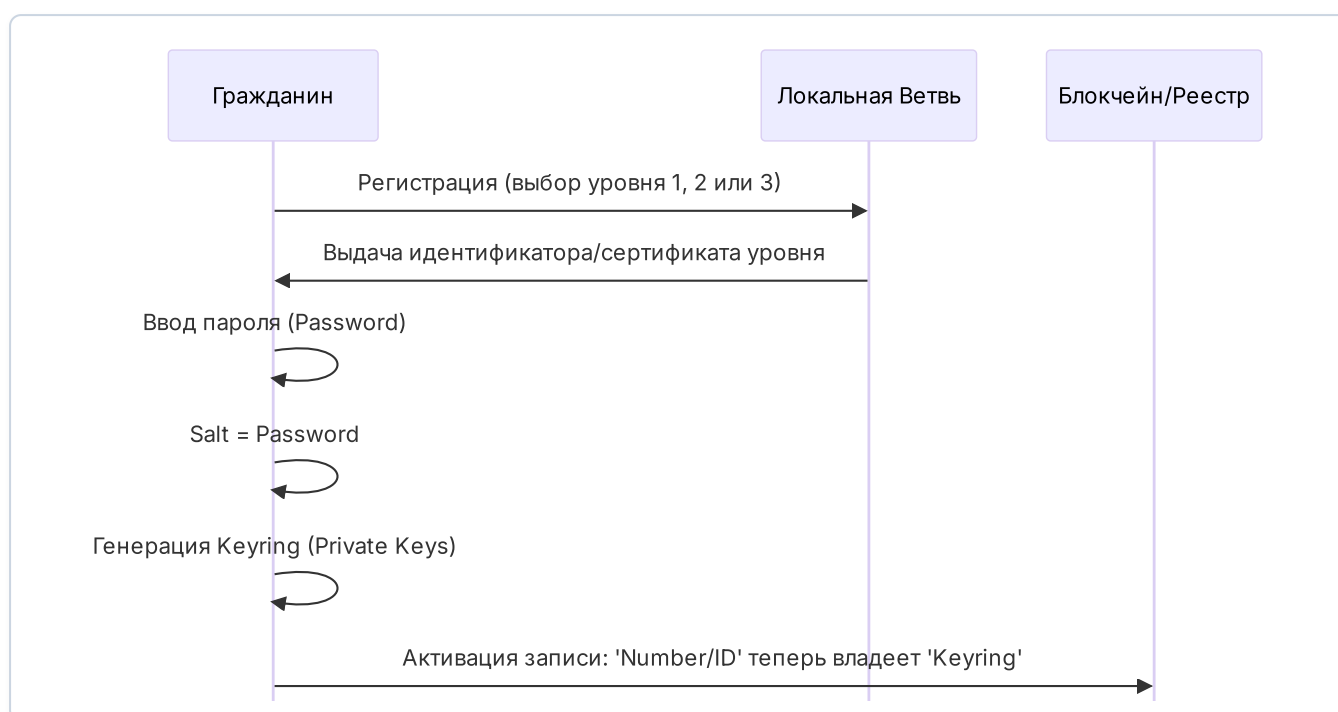
- **Поимённая регистрация (Verified Individual / ГОСТ и ЕСИА):**

- Полная юридически значимая верификация личности через государственную систему ЕСИА / Госуслуги с выпуском квалифицированного сертификата электронной подписи по ГОСТ Р 34.10-2012 / ГОСТ Р 34.11-2012. Требуется для проведения юридически значимых сделок, подписания договоров, голосований собственников МКД и финансовых взаиморасчётов.

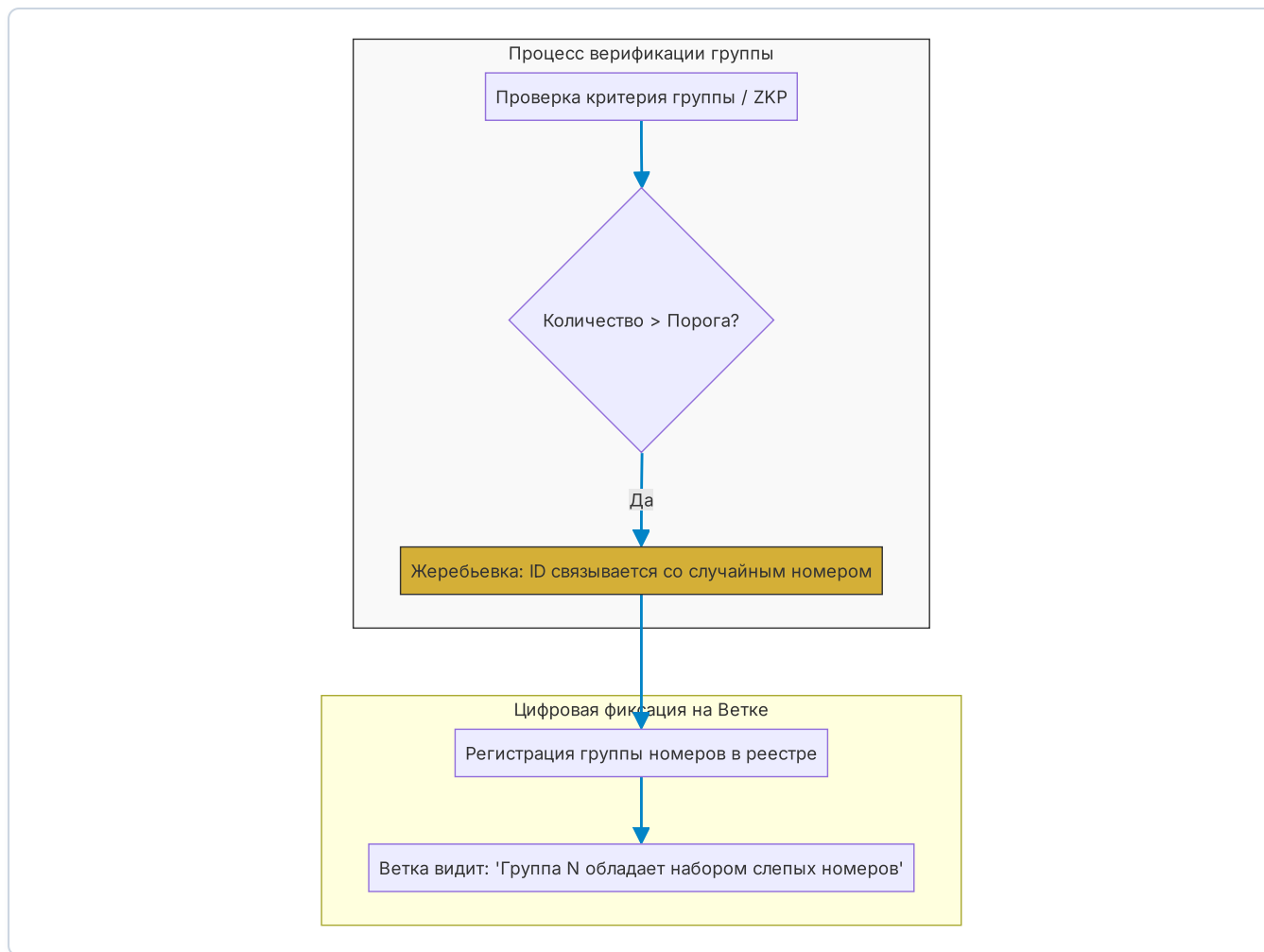
- **Пользователь, регистрируясь по имени, автоматически помещает своё имя в общественный контур.**

Имя связывается с его вкладами и рейтингом и становится видимым в общественных хранилищах и индексах Веток. Перед такой регистрацией приложение объясняет это отдельным экраном, и регистрация завершается только после явного согласия пользователя.

Полностью анонимный рейтинг остаётся за своим анонимным номером и не конвертируется при подключении кошелька (чтобы исключить продажу наработанных анонимных репутаций). Социально анонимный рейтинг, напротив, конвертируется в поимённый статус при раскрытии имени, так как личность гражданина уже подтверждена единственным кошельком в Банке России. Параллельно действует социальная регистрация через очные поручительства соседей без финансовых проверок.



Механизм формирования анонимной группы второго уровня:

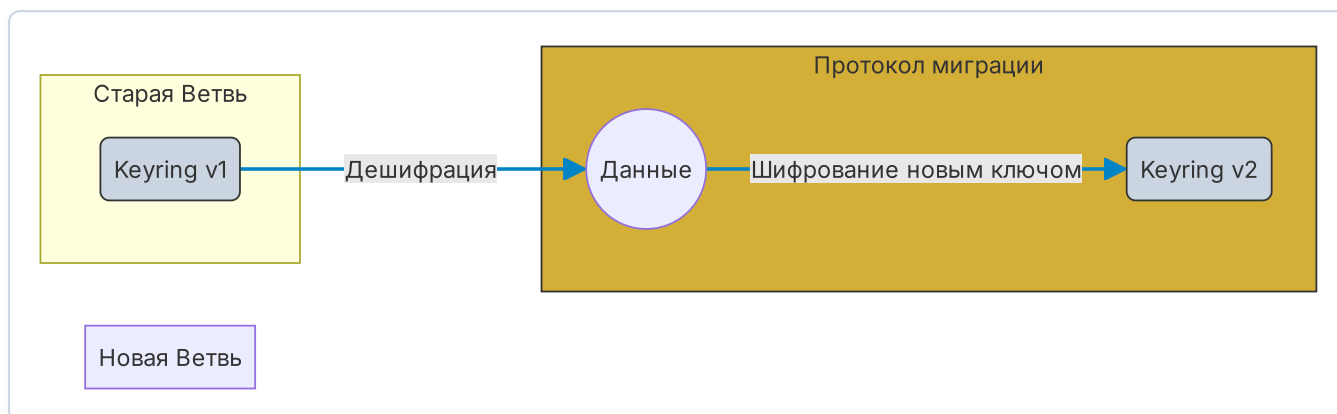


Верифицированные пользователи третьего уровня также могут публично подписывать свои записи, гарантируя участникам экосистемы авторство контента в общественных или совместных хранилищах.

1.8. Аутентификация и миграция

OIDC-авторизация: Для доступа к данным и интероперабельности используется стандарт OpenID Connect, где «Ветвь» выступает доверенным центром аутентификации.

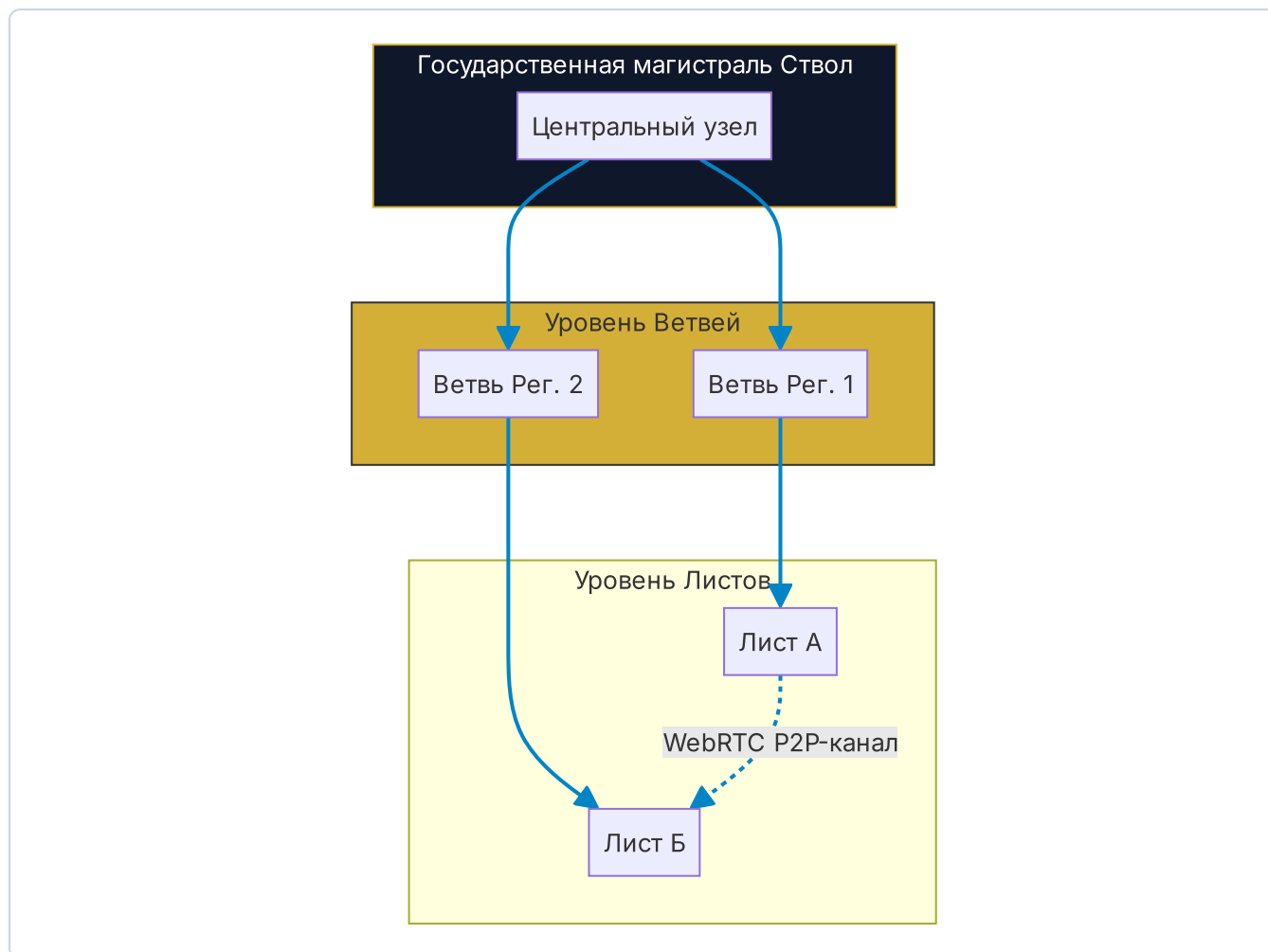
Защита от связывания: При смене локации или ключей пользователь выполняет перешифрование хранилищ (Keyring v1 → Keyring v2). Это гарантирует, что метаданные инфраструктуры старого региона не могут быть использованы для корреляции активности пользователя в новом.



2. Сетевая топология и взаимодействие узлов

2.1. Иерархическая структура сети

Система построена как логическая древовидная иерархия.



Такая топология:

- Ограничивает поверхность атаки: «Ветви» принимают только локальный трафик, предотвращая DDoS-атаки извне.
- Обеспечивает производительность: Быстрый доступ к локальным данным для региональных пользователей.
- Масштабируемость: Каждый уровень иерархии (федеральный округ/область/район) выполняет роль агрегатора для вышестоящего уровня. Данные в системе дублируются «вверх» для аналитики, избавляя от необходимости создавать монолитные узлы.
- Защищенный шлюз и сквозное туннелирование ([PassThroughConnection](#)): Категориальные и отраслевые ветви (например, реестры ЖКХ, кооперативные доски взаимопомощи «Забота») могут физически экранироваться за муниципальной Веткой. Подключение Листа к целевой службе туннелируется через защищенный сквозной шлюз ([PassThroughConnection](#)) с использованием симметричного шифрования ГОСТ Р 34.12-2015 («Кузнечик»). Это полностью скрывает внутреннюю инфраструктуру прикладных ветвей от сканирования и сетевых атак из публичного Интернета.

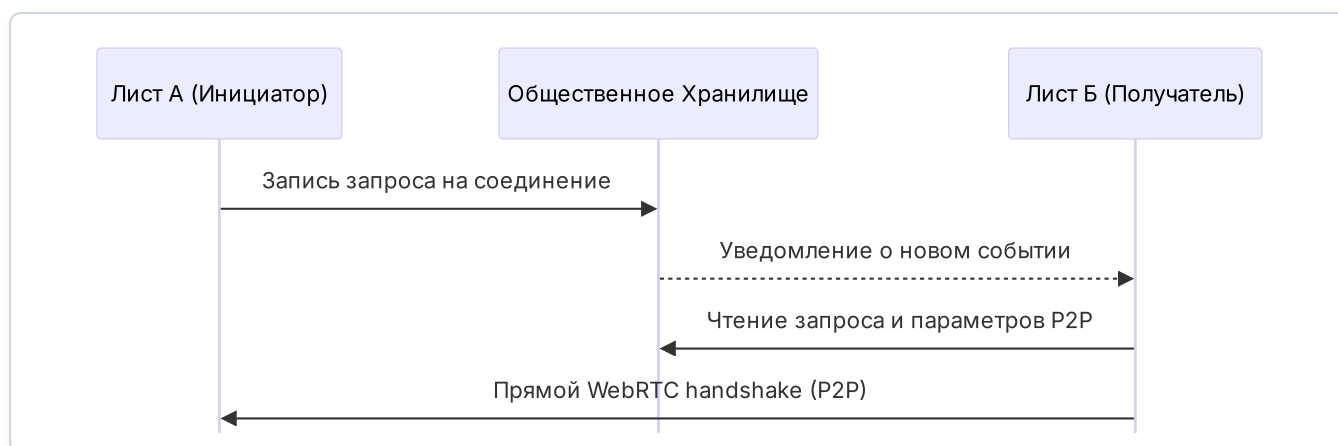
- Гибридная маршрутизация и клиентская реляционная фильтрация: Вместо тяжелых серверных **JOIN** - операций клиентский узел (Лист) загружает легковесные листовые проекции из независимых хранилищ (например, географическое хранилище района и отраслевое хранилище услуг) и выполняет локальные реляционные выборки (**INTERSECT** , **JOIN**) непосредственно в локальной СУБД SQLite/IndexedDB за доли миллисекунды.

2.2. Протокол взаимодействия и P2P-обмен

Хранилище это распределенный сигнальный сервер. Вместо того чтобы полагаться на централизованный или «Ветевой» сервис сигнализации (как в классическом WebRTC), используется сама архитектура хранилищ как асинхронный механизм установления соединения.

Пользователи «общаются» через обновление состояний в общих общественных хранилищах, которые служат своего рода «доской объявлений», видимой для целевого узла. Это избавляет систему от необходимости иметь выделенные STUN/TURN серверы в каждой «Ветви».

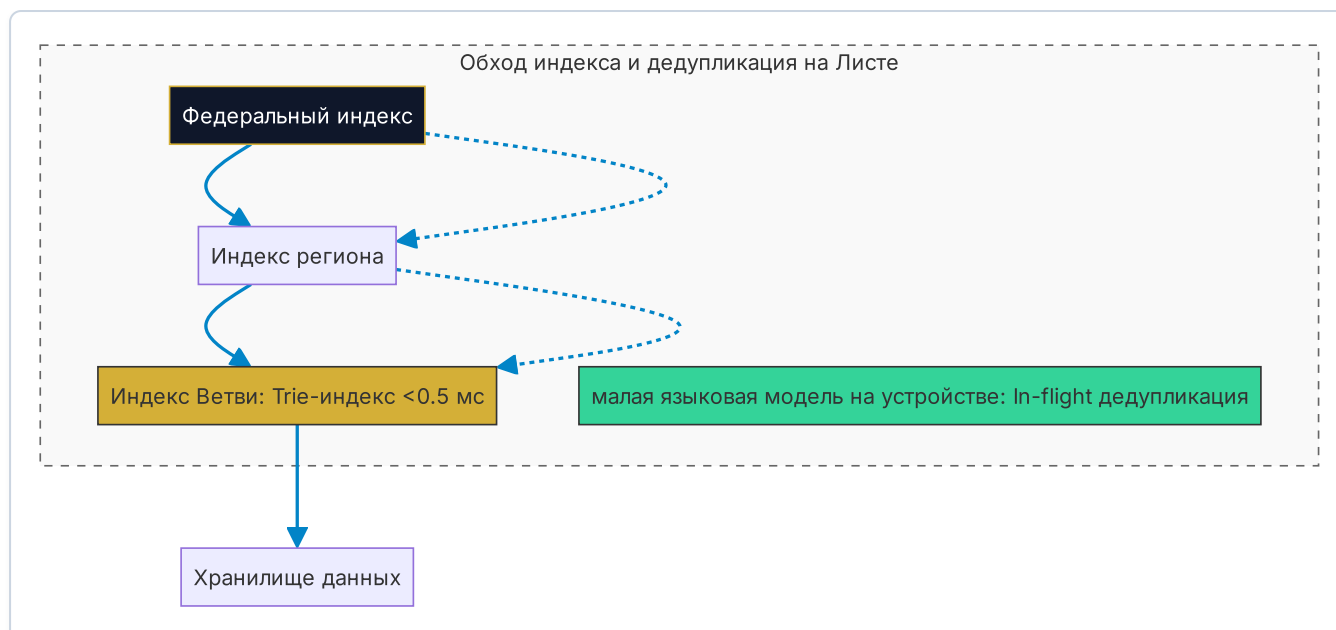
После успешного рукопожатия «Листы» устанавливают прямое WebRTC-соединение, минуя инфраструктуру «Ветвей» для передачи тяжелых данных.



Таким образом, «Ветвь» работает как контрольно-пропускной пункт (плоскость управления), а WebRTC обеспечивает высокоскоростной транспортный уровень (плоскость данных).

2.3. Иерархическое распределенное индексирование и TreeSearch

Поиск данных реализован через рекурсивный спуск по дереву индексов. Каждый узел (от Листа до Ветви) хранит только метаданные для следующего уровня вниз, что делает поиск высокопроизводительным и защищенным.

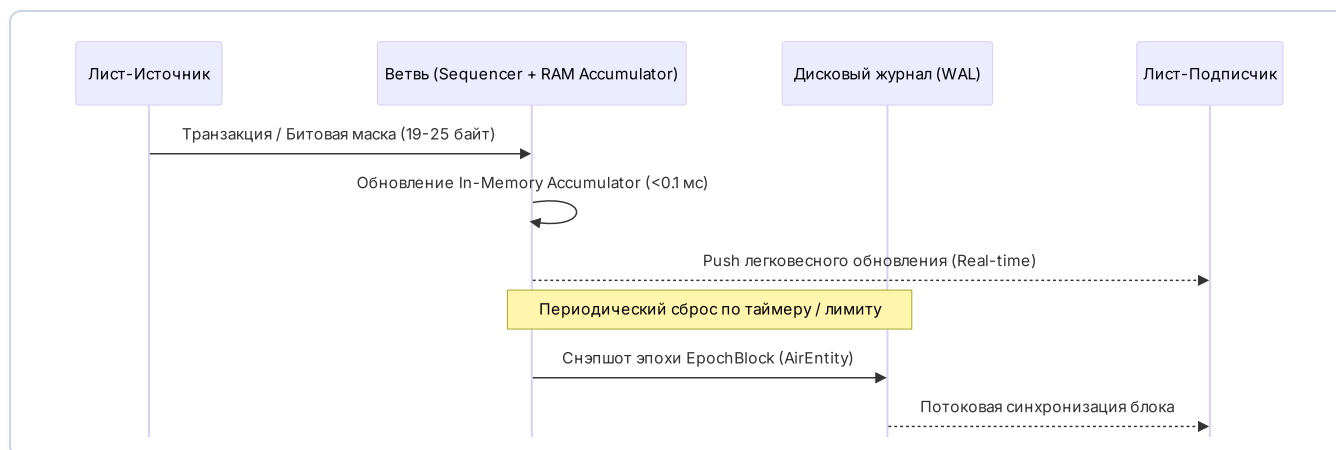


Применяется принцип распределенного индексирования, при котором каждый узел (от Листа до Ветви) хранит лишь необходимый объем метаданных для навигации, а не всю глобальную базу данных. Это превращает поиск по системе в процесс рекурсивного спуска по дереву индексов, что обеспечивает высокую производительность и масштабируемость:

- **Префиксные Trie-индексы на Ветке:** Секвенсор Ветки поддерживает в оперативной памяти (In-Memory) компактный префиксный Trie-индекс по заголовкам и публичным метаданным хранилищ, обеспечивая автодополнение и локальный поиск с субмиллисекундным временем отклика (<0.5 мс).
- **Малая языковая модель на устройстве:** При создании новых топиков, опросов или заявок на устройстве пользователя запускается квантованная локальная нейросеть (SLM), которая анализирует семантику вводимого текста и выполняет дедупликацию «на лету» (in-flight deduplication), предлагая подключиться к уже существующим обсуждениям до отправки данных в сеть и без раскрытия черновика серверу.
- **Отвязка поиска от агрегативных страниц:** Поисковый контур TreeSearch индексирует глобальные заголовки и метаданные сущностей и полностью отвязан от постраничной структуры дочерних записей ([IChildRecordPage](#)), что гарантирует линейную масштабируемость поиска независимо от объема накопленных данных.

2.4. Репликация через Transaction Log, In-Memory аккумуляторы и снимоты EpochBlock

Обновления происходят в реальном времени посредством событийной рассылки записей лога транзакций. «Листы» подписываются на изменения, и новые данные автоматически доставляются («пущатся») вниз по иерархии к заинтересованным пользователям. «Лист» получает только то, на что он подписан, и ровно в тот момент, когда данные обновляются на «Ветви».



Для достижения экстремальной производительности и защиты от перегрузки конвейер Ветки использует трехуровневый механизм обработки:

- **In-Memory Accumulators (аккумуляторы в оперативной памяти):** Для высокочастотных операций (учет голосов в опросах, фиксация реакций и отметок благодарности) Ветка не производит дисковую запись на каждую транзакцию. Изменения аккумулируются непосредственно в RAM секвенсора в виде компактных битовых масок и счетчиков (19–25 байт на запись).
- **Снэпшоты эпохи (`EpochBlock extends AirEntity`):** Накопленные состояния периодически (по временному интервалу или при достижении порога изменений) сбрасываются в дисковый журнал предзаписи (WAL) единым блоком-снэпшотом `EpochBlock`, наследующим системный контракт `AirEntity`. Это снижает дисковую нагрузку (I/O) более чем на 99.8% и обеспечивает мгновенное восстановление узла при перезапуске.
- **Динамические квоты (`user_factor_quotas`):** Ветка рассчитывает доступную пропускную способность для каждого подключенного устройства на основе его коэффициента верификации (`user_factor`). Это защищает инфраструктуру от флуда и спам-атак со стороны неавторизованных клиентов.

2.5. Физическая изоляция (Runtime Isolation) и архитектура Zero-Rendering

Для обеспечения максимальной стабильности и безопасности каждая «Ветвь» работает в изолированном контейнерном/виртуальном контуре. Это исключает риск «утечки памяти» (Memory Leakage) и конфликтов ресурсов:

- **Архитектура Zero-Rendering (нулевой серверный рендеринг):** Серверы Ветки функционируют строго в безголовом (headless) режиме потоковой обработки бинарных дельт. На стороне Ветки выполняется **0% рендеринга HTML или DOM**. Вся вычислительная нагрузка по построению пользовательского интерфейса, парсингу шаблонов, верстке и виртуализации длинных списков на 100% перенесена на графические процессоры (GPU) оконечных устройств (Листов). Это позволяет одной компактной Ветке обслуживать десятки тысяч одновременных соединений при минимальных аппаратных требованиях к серверу.
- **Изоляция процессов:** Аналитическая обработка данных и избыточность обеспечиваются за счет дублирования логов «вверх» по иерархии, а не за счет объединения разнородных процессов внутри одного монолитного сервера.

3. Архитектура хранилищ и данных

3.1. Хранилище как блокчейн-объект

Каждое хранилище функционирует как автономный блокчейн. Транзакции подписываются ключевой парой владельца, что гарантирует:

Неизменность (Immutability): Никто, включая инфраструктуру «Ветвей», не может подделать данные.

Верифицируемость: «Лист» проверяет каждую входящую транзакцию, обеспечивая подлинность контента.

Конвергенция (Reconciliation): При работе в офлайн-режиме (разветвление данных) блокчейн-структура позволяет безошибочно объединить (reconcile) состояния всех «Листов» при восстановлении связи.

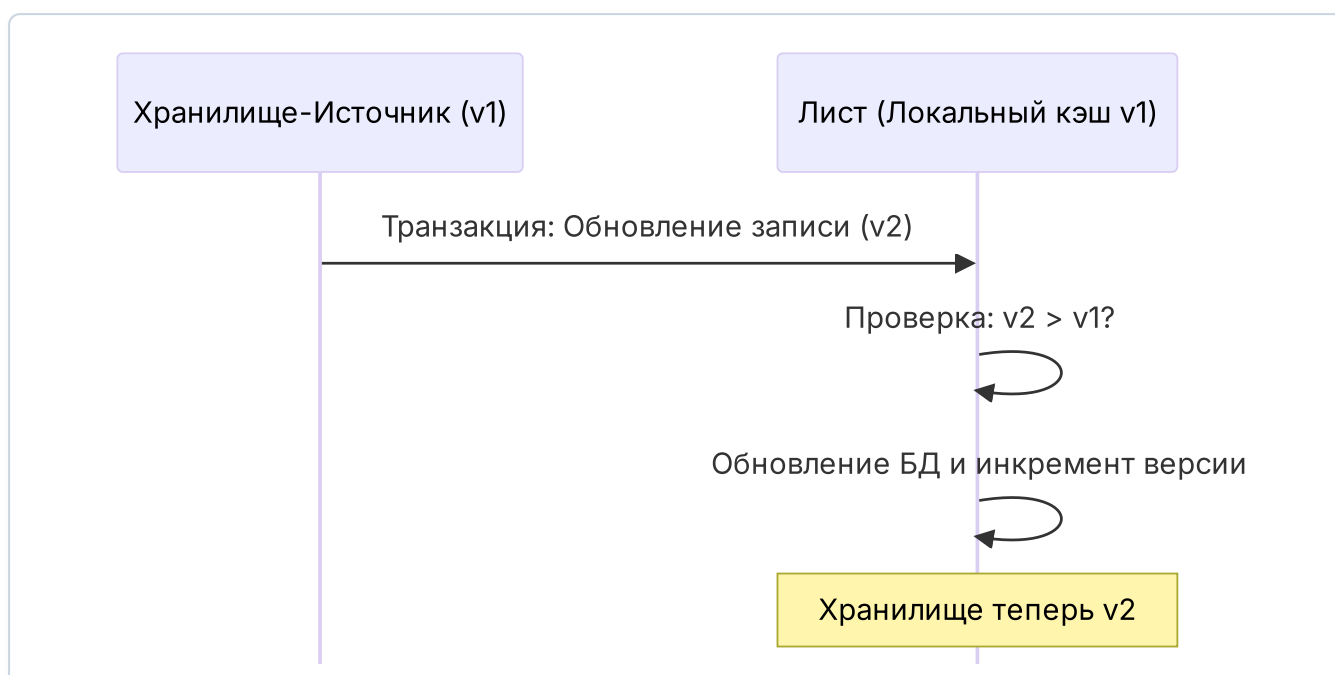
3.2. Согласованность данных через версионные лог-записи

Система использует идемпотентное обновление:

Версионность: Каждая копия записи в кэше хранилища сопровождается номером версии.

Автоматическое обновление: При получении нового лога транзакций «Лист» сверяет версии. Если версия новее, локальная БД обновляется.

Идемпотентность: Повторная обработка одного и того же обновления от разных «Листов» не вызывает избыточных операций, так как состояние БД уже соответствует актуальной версии.



Каждая хранилища это «активный объект», который знает свою версию и автоматически обновляется при получении новых данных из потока транзакций без централизованной «блокировки» или протокола двухфазной фиксации (2PC).

3.3. Динамическое управление данными

Виртуализация: Хранилища существуют как «объекты-ссылки», которые активируются только в контексте выполнения задачи.

Авто-загрузка зависимостей: «Лист» отслеживает связи между схемами и автоматически подгружает связанные хранилища. Это системный кэш данных с ленивой загрузкой (Lazy Loading), где «Лист» сам управляет графом зависимостей между хранилищами, подгружая их в оперативную реляционную БД по мере необходимости и выгружая их, когда они становятся неактуальными.

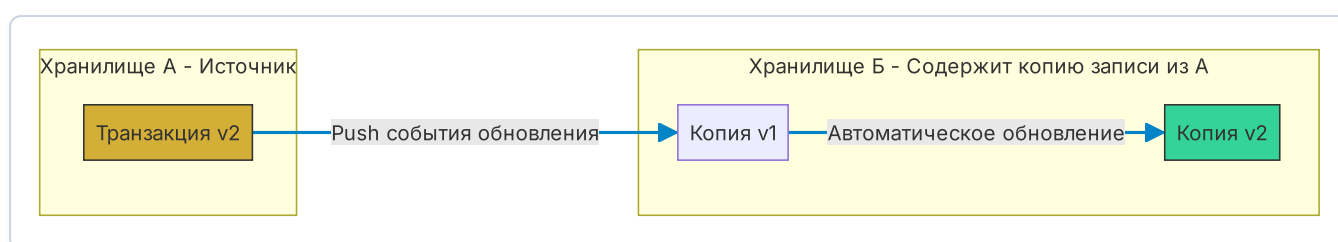
Управление памятью: IndexedDB используется как «холодное» хранилище для зашифрованных логов транзакций, из которых «Лист» восстанавливает состояние БД в момент обращения (re-hydration).

3.4. Кэширование ссылочной целостности

В каждое хранилище встраивается копия связанных записей. Это создает «гравитационный эффект» данных: основное хранилище притягивает контекст, необходимый для его обработки, минимизируя потребность в подгрузке зависимостей при каждом обращении.

Это механизм локального кэширования связей. Используется модель денормализации через кэширование ссылок (копирование связанных записей непосредственно в хранилище), что превращает хранилище в самодостаточный объект (self-contained object). Это позволяет просматривать данные сразу — все «соседние» данные уже под рукой. Одновременно, в зависимости от настройки Листа все хранилища использующегося загруженного хранилища могут быть подзагружены до того как пользователь обратится к ним.

Копированные связанные записи обновляются по мере обновления ссылаемых хранилищ.



3.5. Декомпозиция связей «Один-ко-Многим» и Агрегативные Хранилища страниц (IChildRecordPage)

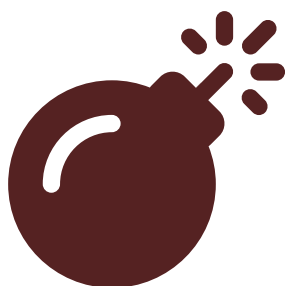
В традиционных архитектурах баз данных связи «Один-ко-Многим» (1:N) представляют собой фундаментальную проблему масштабирования: при лавинообразном росте дочерних записей (комментарии к посту, голоса в опросе, отклики на заявку помощи) единая таблица деградирует по скорости выборки, а монолитный документ превышает лимиты репликации и оперативной памяти клиентских устройств.

Платформа «Турбаза» решает эту проблему через механизм **самозапечатывающихся агрегативных страниц (IChildRecordPage)**:

- **Декомпозиция на страницы фиксированной емкости (~1000 элементов):** Связь 1:N декомпозируется не в бесконечный монолитный лог, а в последовательность изолированных агрегативных хранилищ страниц. Каждая страница вмещает приблизительно 1000 дочерних записей.
- **Листовые проекции (Shallow Leaf Projections):** Агрегативная страница не хранит тяжелые тела дочерних сущностей. В ней сохраняются исключительно компактные проекции, достаточные для мгновенного рендеринга и виртуализации списков в пользовательском интерфейсе:
 - `childEntityId` (идентификатор сущности);
 - `childRepoId` (хранилище сущности);
 - `title` / краткое превью (для отображения в UI);
 - `createdAt` (временная метка);
 - `status` / числовой флаг.

Полные тела сущностей хранятся в их собственных независимых хранилищах и загружаются лениво (по клику пользователя).

- **Каскадное распространение обновлений проекций (`childRecordUpdate`)**: При изменении названия (`title`) или статуса сущности в ее основном хранилище секвенсор Ветки формирует и рассылает легковесную дельту `childRecordUpdate` в соответствующую агрегативную страницу, гарантируя актуальность отображения списков без необходимости полной перезагрузки данных.
- **Автономное запечатывание и вечное кэширование**: По достижении лимита (~1000 записей) страница криптографически запечатывается:
- Секвенсор Ветки переводит страницу в статус неизменяемой (Read-Only) и переключает входящий поток записей на новую открытую страницу (`currentActivePageRepoId`);
- Запечатанные страницы обладают 100% иммутабельностью, что позволяет навечно кэшировать их на периферийных узлах CDN и в IndexedDB клиентских Листов, полностью исключая повторный сетевой трафик при пагинации истории;
- Запечатыванием управляет секвенсор Ветки, а Листы лишь подписывают дельты для текущей открытой страницы, не требуя сложной распределенной координации.



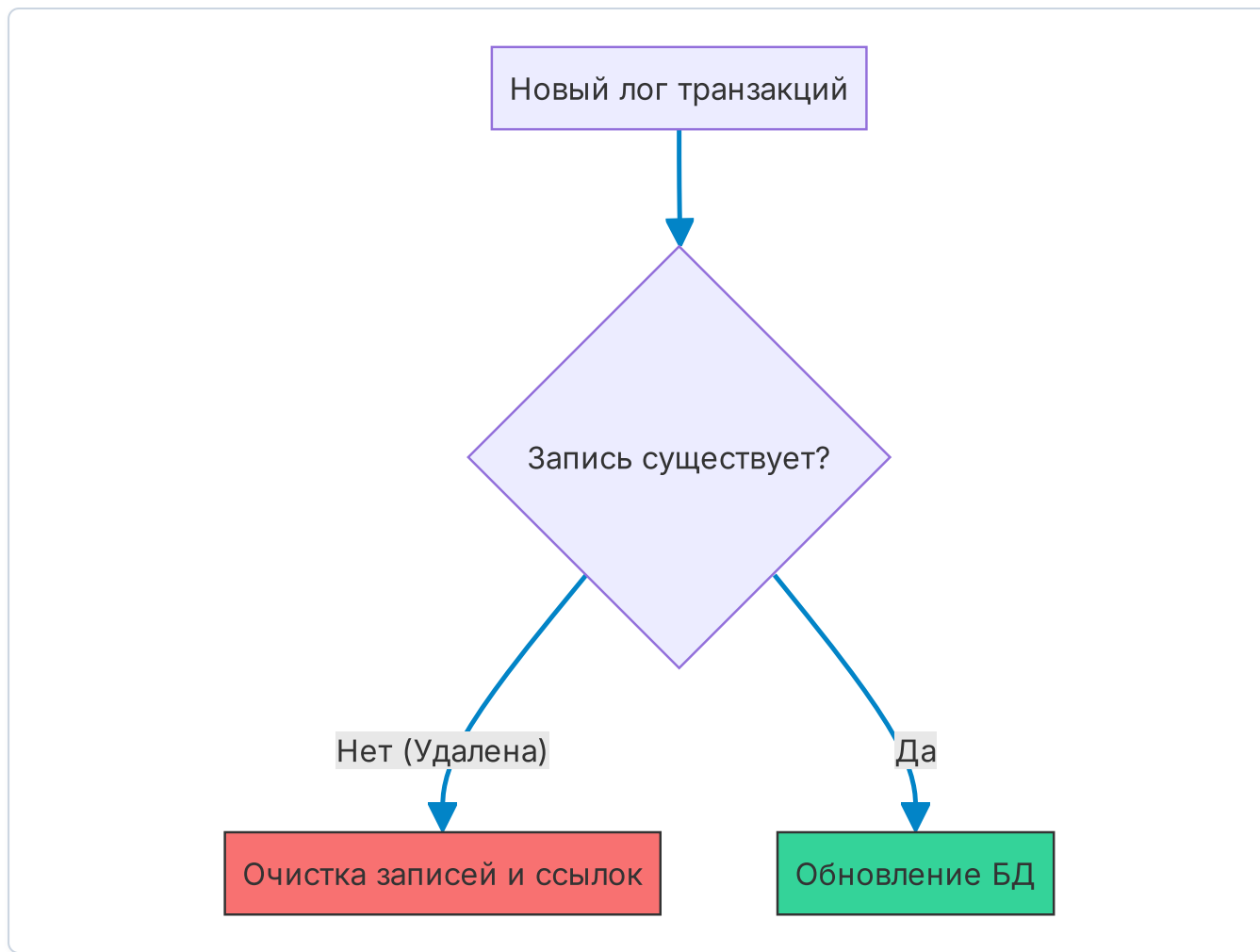
Syntax error in text mermaid version 10.9.8

3.6. Управление жизненным циклом и очистка

Система использует идемпотентное обновление. Удаление или модификация записей происходит путем добавления новых транзакций, а оптимизация локальной БД («pruning») выполняется «Листами» при загрузке новых блоков транзакций, удаляя записи, которые больше не являются актуальными в текущем состоянии блокчейна.

Активное удаление: При обработке нового транзакционного лога система выполняет проверку актуальности. Если запись помечена как удаленная или неактуальная (pruned), она изымается из активной реляционной БД.

Оптимизация связей: Если запись удаляется, «Лист» автоматически очищает (prune) все зависимости и ссылки в связанных хранилищах, предотвращая наличие «битых» связей (broken references).



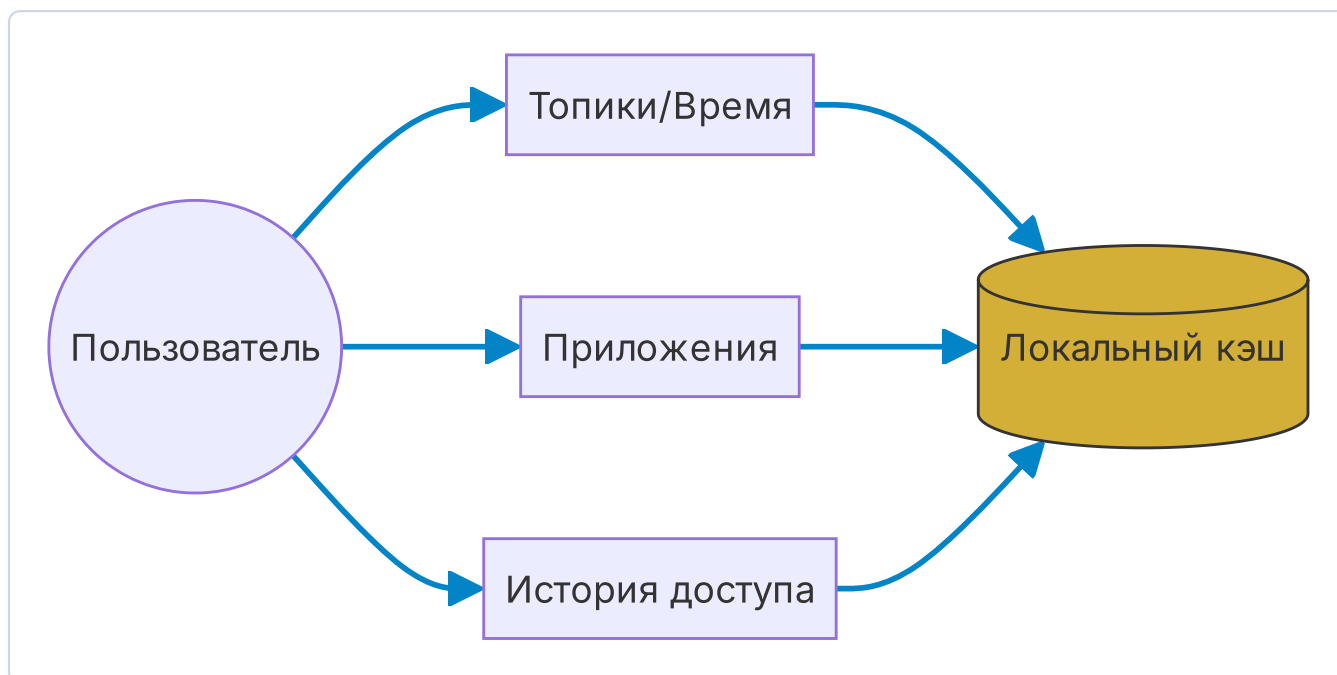
3.7. Многослойная система индексов

Система поддерживает три уровня навигации:

Топики/Хронология: Статичная иерархия для глубокого поиска.

Приложения: Индексы по логике взаимодействия.

История доступа: Динамический «легкий» индекс для быстрого возврата к недавним данным.



4. Управление ключами и безопасность

4.1. Keyring как доверенный объект

Ключи не являются «системными файлами»; они живут в Keyring Repository. Ключ — это не просто статичный файл, а объект в хранилища (Keyring), который может быть разделен между участниками группы.

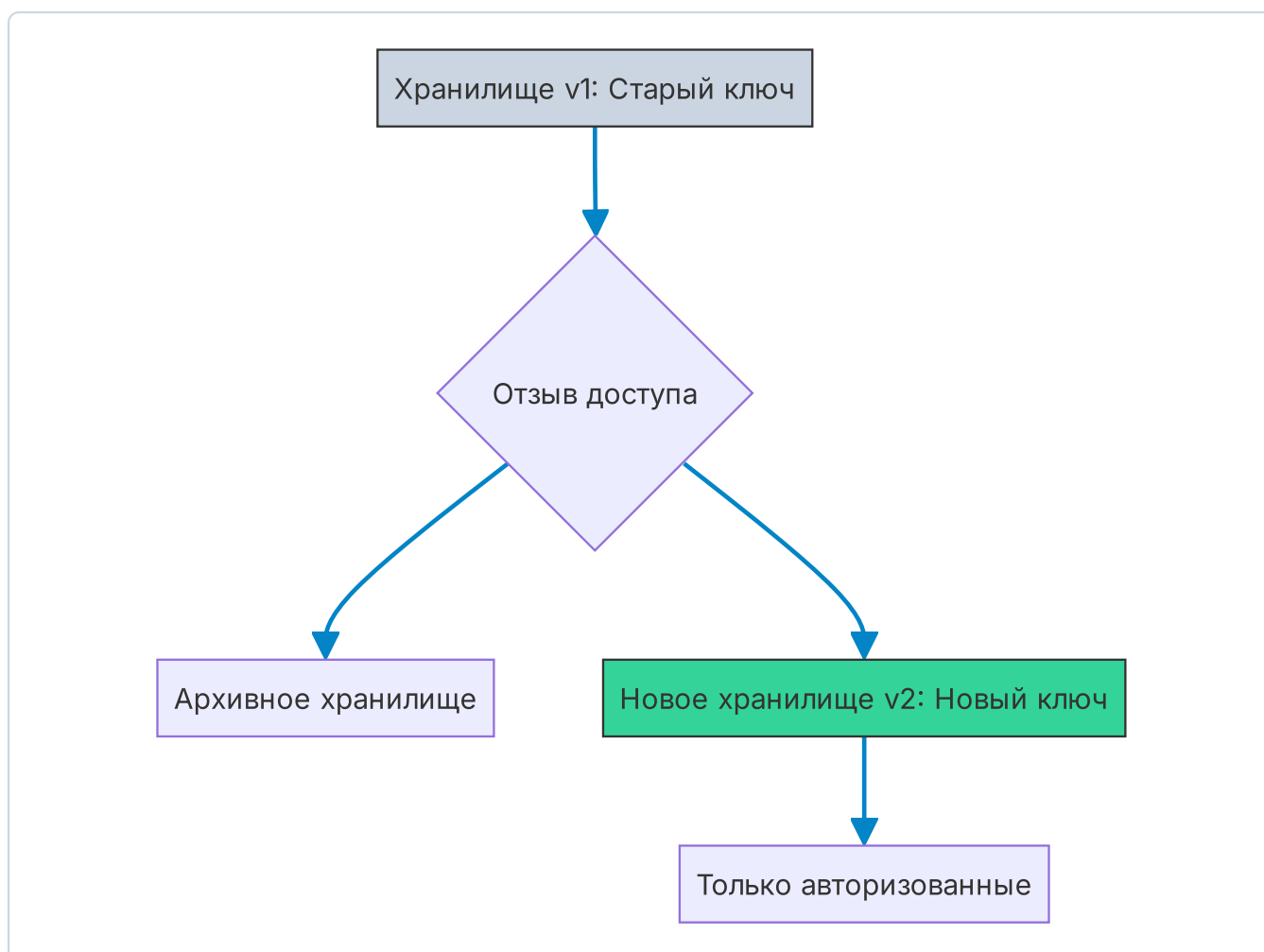
Это позволяет:

Делегировать доступ: Копирование Keyring между устройствами или доверенными членами группы (семья/коллеги).

Синхронизировать доверие: Использование Keyring как объекта в сети хранилищ, который подчиняется общим правилам распространения логов.

4.2. Безопасность через ветвление (Security Forking)

Вместо попыток «отозвать» ключ у скомпрометированного участника, система использует форкинг хранилищ. При изменении состава группы старое хранилище «замораживается», а данные переносятся в новый контейнер с новым ключом. Это гарантирует математически чистое разделение доступа.



Создание Fork-хранилища: Система генерирует новое хранилище с обновленными правами доступа и новым шифрующим ключом.

Перенос связей: Существующая история (до момента разделения) сохраняется для всех участников, но доступ к будущим «веткам» транзакций получают только авторизованные пользователи.

Разрыв цепочки: Старое хранилище становится «архивным» для группы; новые данные в него не добавляются.

4.3. Автоматизированная ротация ключей

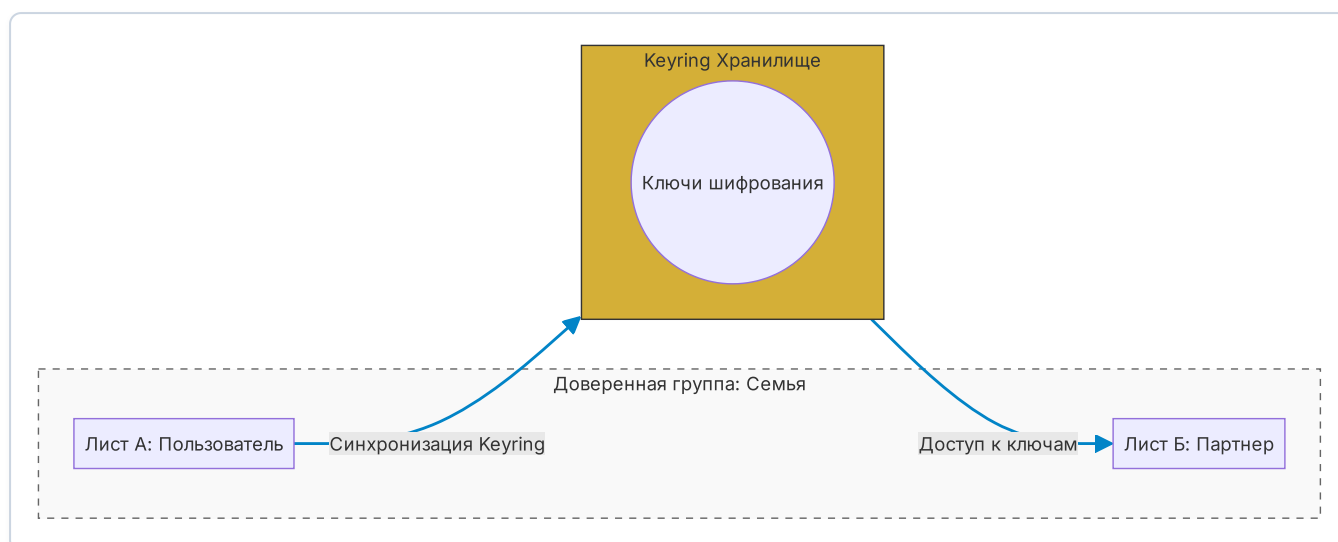
Для минимизации рисков компрометации система поддерживает:

Default Automatic Rotation: Периодическое обновление ключей в Keyring для поддержания высокого уровня безопасности без участия пользователя.

On-Demand Manual Rotation: Возможность принудительного обновления ключей по требованию владельца.

4.4. Модель делегированного доступа (Keyring Sharing)

Keyring переходит от одного «Листа» к другому через доверенные хранилища, позволяя членам группы (например, семье) иметь синхронизированный доступ к одним и тем же данным.



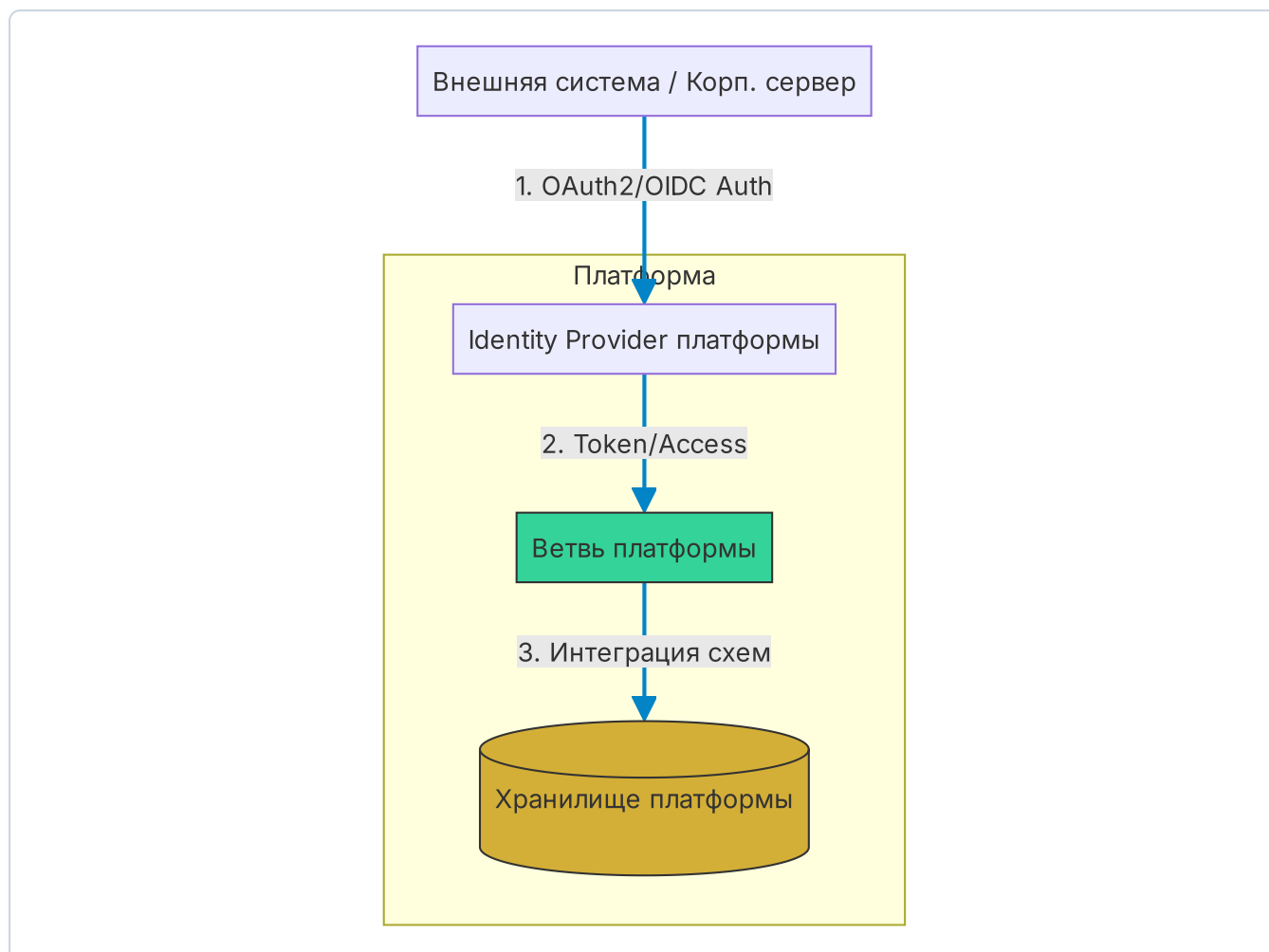
Потеря доступа не становится катастрофой, если есть «доверенные узлы» (члены семьи или коллеги), у которых есть копия ключей.

Часть II: Безопасность, изоляция и соблюдение требований закона

5. Архитектура внешнего взаимодействия

5.1. Интеграция через OpenID Connect (OIDC)

Платформа использует стандарт OIDC для первичной аутентификации всех узлов, включая корпоративные серверы. Это позволяет внешним системам выступать в роли «Листов» или «Ветвей», сохраняя при этом безопасность через привязку к Keyring-хранилищу.

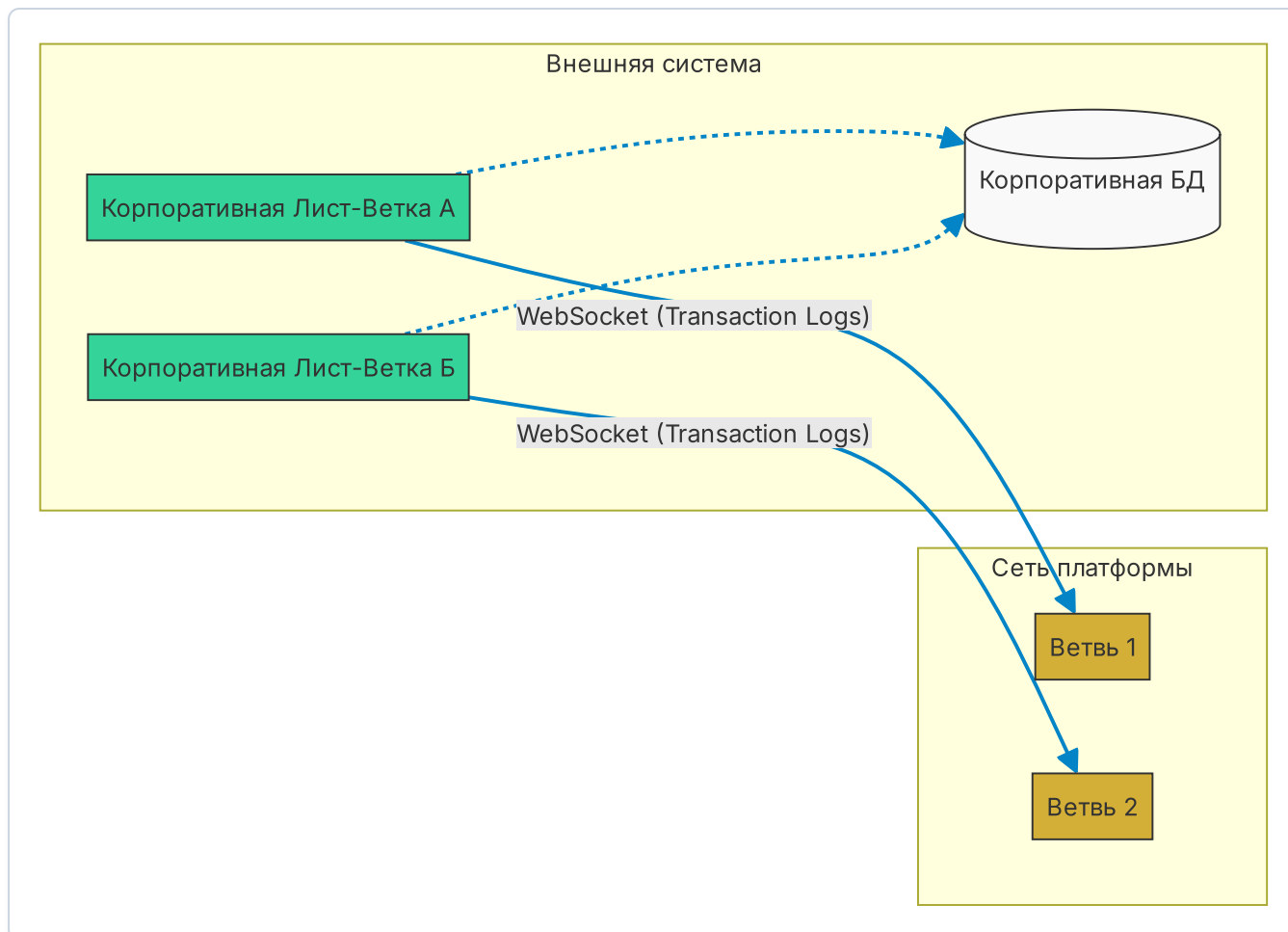


Любой внешний узел (корпоративный сервер или стороннее приложение) проходит аутентификацию через OIDC для входа в сеть. После успешного подтверждения личности он получает статус «Листа» или «Ветви» в зависимости от своих задач и полномочий.

Внешний узел является «Листом» если он работает с государственными хранилищами и является «Веткой» если он позволяет государству работать со своими хранилищами.

5.2. Синхронизация через WebSocket

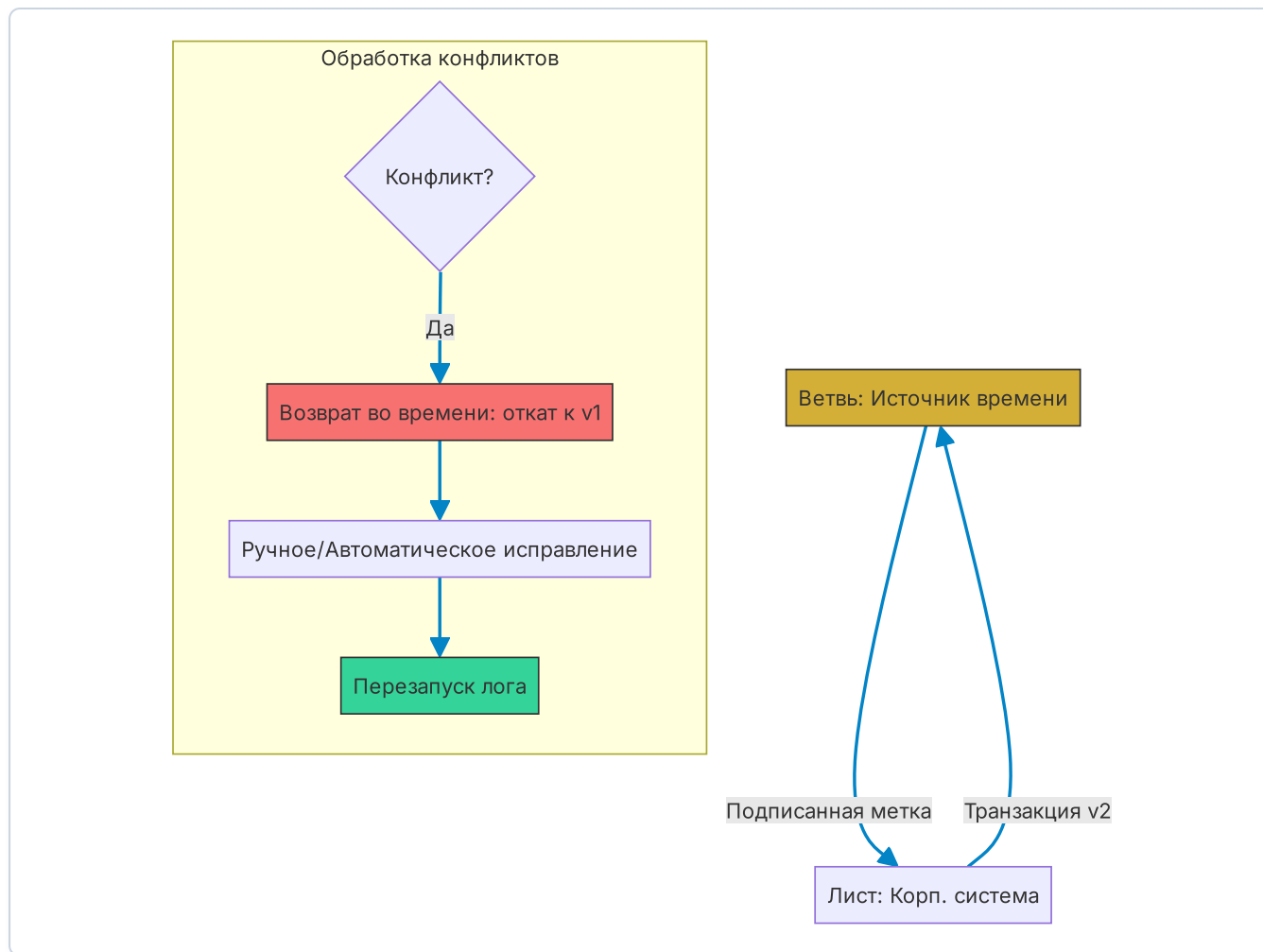
Для обеспечения работы в реальном времени, внешние системы поддерживают постоянное WebSocket-соединение с «Ветвью». Это позволяет автоматически получать новые записи лога транзакций без необходимости постоянного опроса (polling).



Сторонняя система развертывает один или несколько «Листов», каждый из которых держит постоянный WebSocket-канал с соответствующей «Ветвью». Это превращает внешнюю БД в динамический участник сети, который автоматически актуализирует свои данные при любых изменениях в хранилищах платформы.

5.3. Разрешение конфликтов и путешествия во времени

Система использует иерархический арбитраж времени: «Ветвь» рассылает подписанные временные метки, которые «Листы» используют для подписи своих транзакций. В случае конфликта (например, одновременной записи) применяется политика Last-Write-Wins (последний по времени побеждает), а для исправления ошибок используется концепция «путешествия во времени» — откат к точке до конфликта и очистка лога.



«Ветвь» выступает как «арбитр времени», синхронизируя все «Листы» через подписываемые временные метки, а окончательное решение конфликтов остается за пользователем, который может «отмотать» состояние БД и внести правки.

6. Механизмы синхронизации и оффлайн-режим

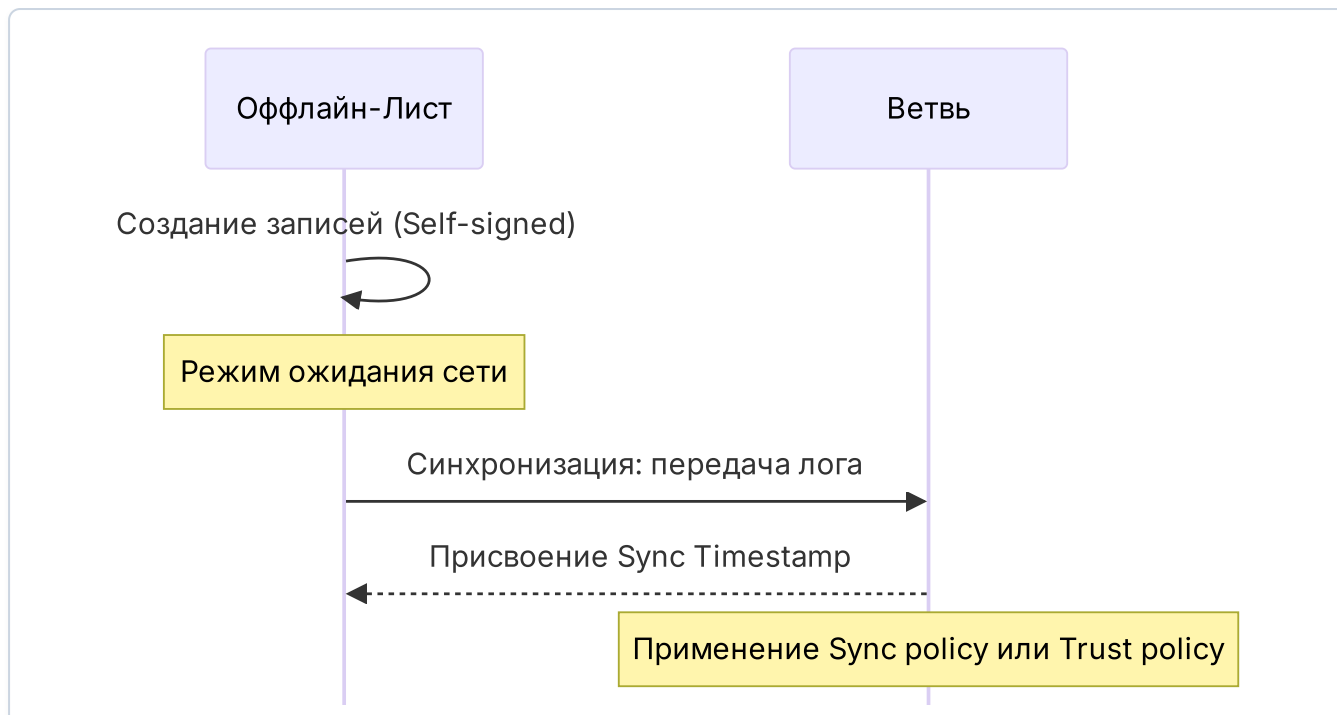
6.1. Политики синхронизации

Система поддерживает гибкие подходы к синхронизации в зависимости от критичности данных:

Sync Timestamp Policy (синхронизация по времени «Ветви»): Используется для высокочиточных данных (например, финансовых транзакций). Локальные записи «Листа» привязываются к временной шкале «Ветви», обеспечивая единый объективный порядок событий.

Trust Offline Leaf Policy (использование локальных меток): Используется для персональных или общественных данных. Система доверяет локальной последовательности событий (self-signed timestamps), что позволяет работать полностью автономно без привязки к глобальному времени «Ствола». Применяется для персональных или общественных данных (форумы, заметки), где приоритет — доступность и скорость.

Если два «Листа» синхронизируются в одну точку, алгоритм внутри хранилища гарантирует, что порядок записей будет вычислен одинаково на всех узлах, избегая «разночтений» в истории.



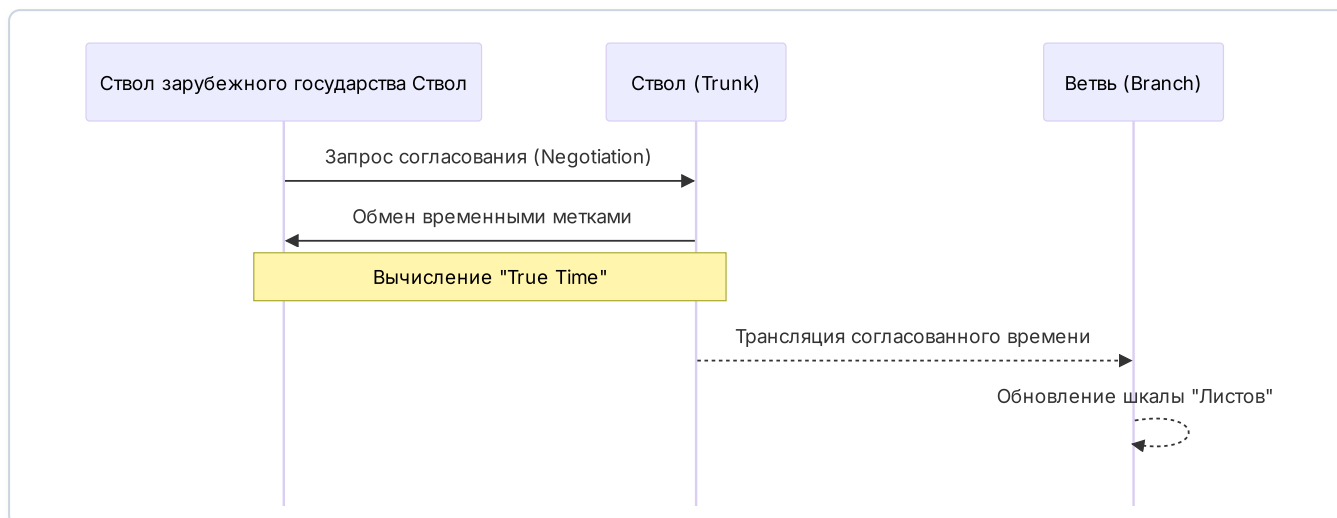
6.2. Иерархическое согласование «Истинного времени»

Для поддержания единой временной шкалы в межгосударственной сети используется протокол многостороннего согласования между корневыми узлами — Стволами (Trunks).

Стволы: Участвуют в межузловом протоколе для вычисления «True Time».

Ветви (Branches): Получают эталонное время от своих родительских «Стволов», обеспечивая согласованность для всех подчиненных «Листов».

Если «Стволы» фиксируют расхождение, они обмениваются серией сообщений для вычисления общего «времени согласия» (True Time), которое затем транслируется вниз по иерархии к «Ветвям» и «Листам».

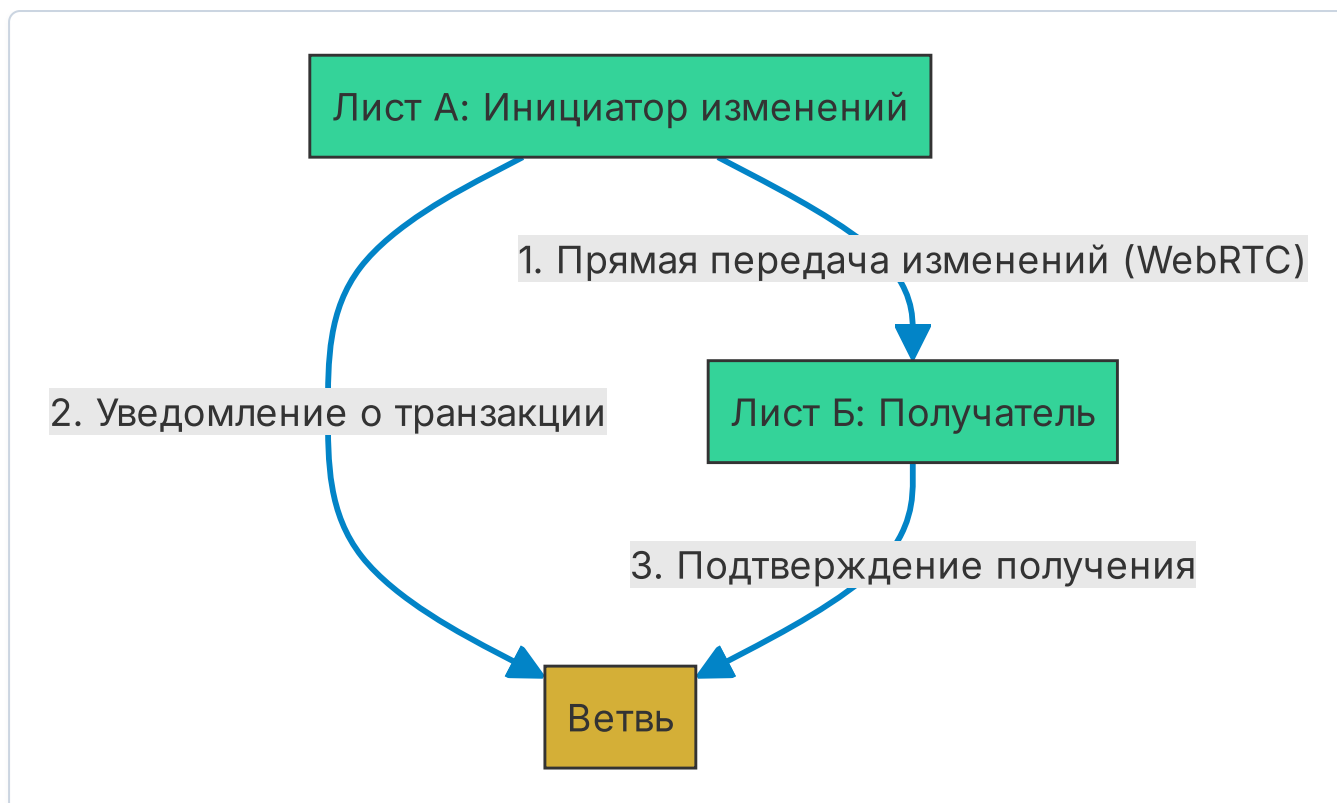


6.3. Гибридная синхронизация (Delta & P2P)

Для оптимизации трафика и работы в реальном времени используется двухслойная передача данных:

WebRTC P2P-синхронизация: «Листы» обмениваются изменениями напрямую, уведомляя «Ветви» лишь о факте успешной доставки. Это снижает нагрузку на центральную инфраструктуру.

Инкрементальный Push: Узлы передают на «Листы» только записи транзакционных логов хранилищ, не имеющие подтверждения синхронизации.



Используется гибридная модель передачи данных, которая сочетает централизованный контроль (через «Стволы» и «Ветви») с децентрализованной доставкой (через WebRTC между «Листами»). Это минимизирует нагрузку на инфраструктуру и обеспечивает невероятную скорость работы, так как «Ветка» не должна отправлять данные на остальные «Листы» если она уже получила подтверждение синхронизации между «Листами» и данные уже были доставлены напрямую.

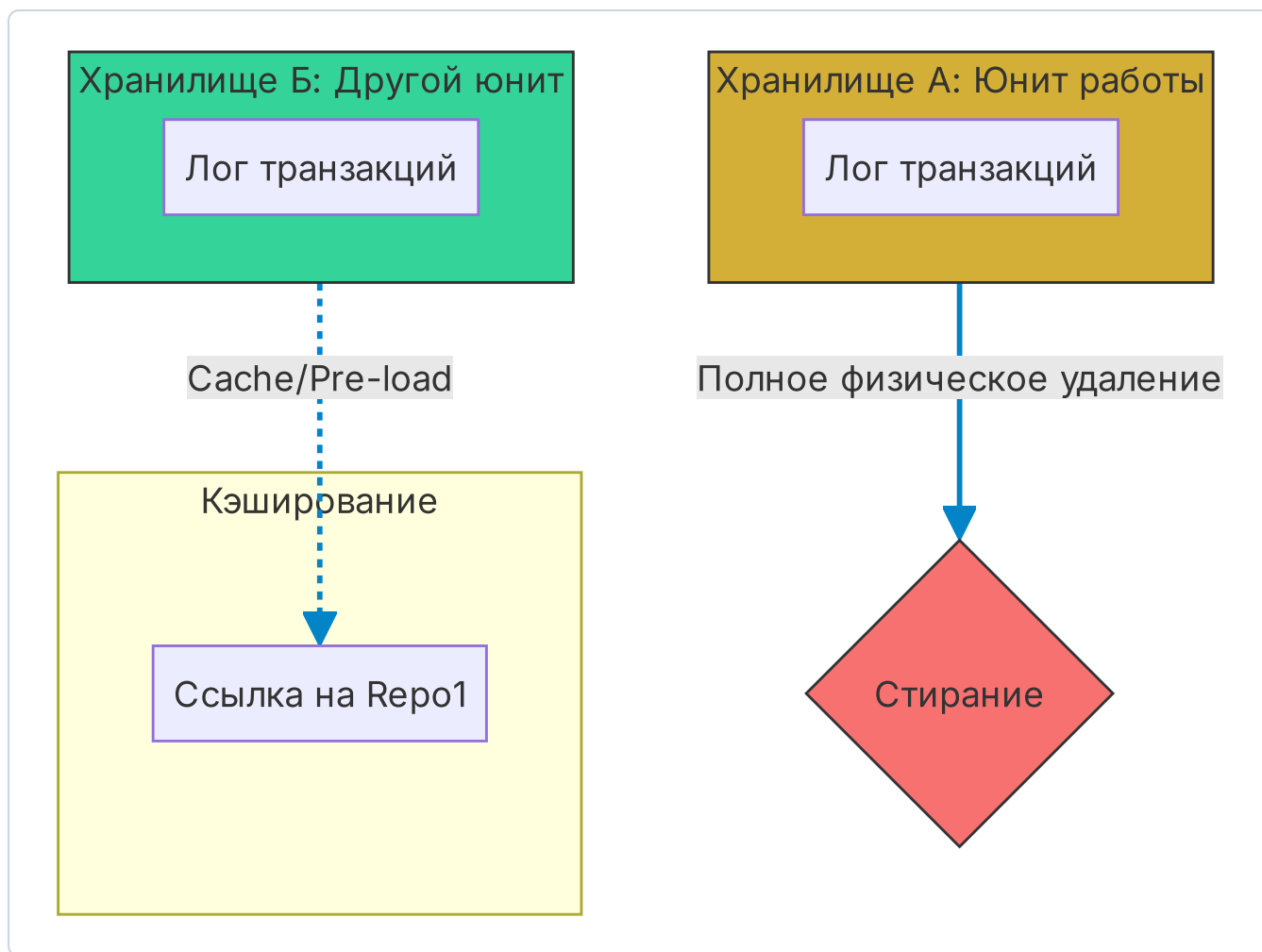
7. Соблюдение требований регуляторов, атомарность и жизненный цикл данных

12.1. Независимые блокчейны-хранилища и «Право на забвение»

В отличие от монолитных блокчейнов, архитектура системы базируется на множестве независимых блокчейнов-хранилищ. Каждый из них представляет собой самостоятельный «юнит работы» (unit of work), что обеспечивает уникальную гибкость в управлении данными.

Физическая изоляция: Поскольку каждое хранилище является отдельным блокчейном без связей с другими, удаление истории транзакций одного хранилища не нарушает целостность системы.

Право на забвение: Соответствие юридическим нормам (например, GDPR) достигается через физическое удаление конкретного хранилища на всех Ветвях и Листах, где он был сохранен. Это гарантирует полное уничтожение данных без необходимости «вырезания» их из глобального лога.



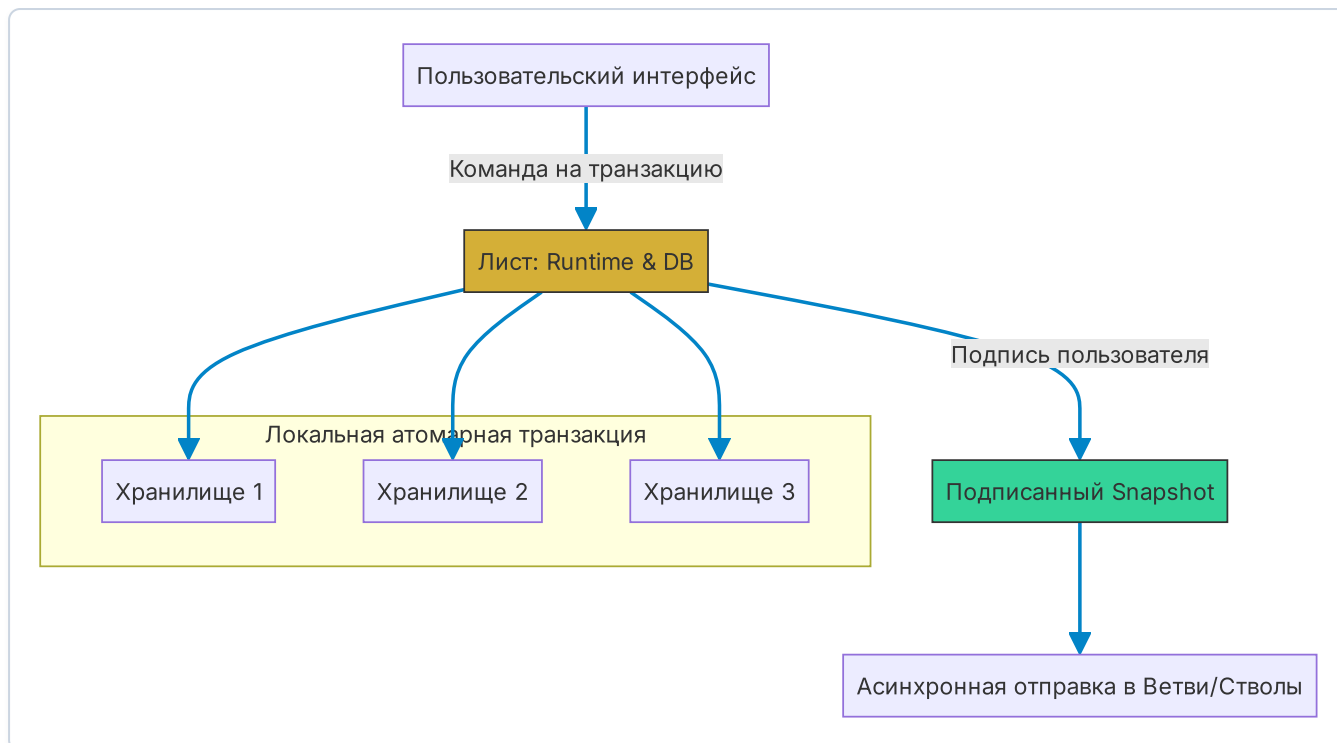
Независимость хранилищ: Позволяет соблюдать «Право на забвение» путем удаления конкретных юнитов работы (хранилищ).

12.2. Атомарность транзакций на уровне Листа

Система исключает необходимость в сложных протоколах распределенных транзакций (например, 2PC), перенося ответственность за их консистентность на уровень «Листа».

Лист как Runtime: Лист является изолированной доверенной средой исполнения (Runtime), объединяющей UI, API-вызовы и базы данных (хранилищ), он фактически является локальным транзакционным менеджером. «Лист» всегда знает конечное состояние всех измененных хранилищ перед их отправкой в цепочки Ветвей/Стволов.

Атомарность: Все изменения в различных хранилищах, инициированные одной операцией, фиксируются как единый «снимок» (snapshot). Транзакция подписывается пользователем локально, гарантируя, что все изменения либо будут приняты одновременно, либо не будут приняты вовсе.



Лист работает как изолированный контейнер состояний. Все изменения в различных хранилищах (Кошелек, История, API-логи) происходят в рамках одного «снимка» (snapshot) и подписываются пользователем как единое целое.

Аудируемость: Каждая транзакция, подписанная на Листе, является юридически значимым актом владения и изменения данных.

12.3. Юридическая значимость и аудит

Аудируемость: Каждая транзакция, подписанная на Листе, является юридически значимым актом владения и изменения данных.

Управление рисками: Разработчики приложений могут структурировать схемы данных, предотвращая утечку чувствительной информации в «gateway-рекорды», при этом сохраняя возможности для эффективного кэширования и предварительной загрузки.

Compliance: Государственные органы могут аудировать конкретные хранилища, не имея доступа ко всей экосистеме, что упрощает прохождение проверок.

Часть III: Программное Обеспечение Листа

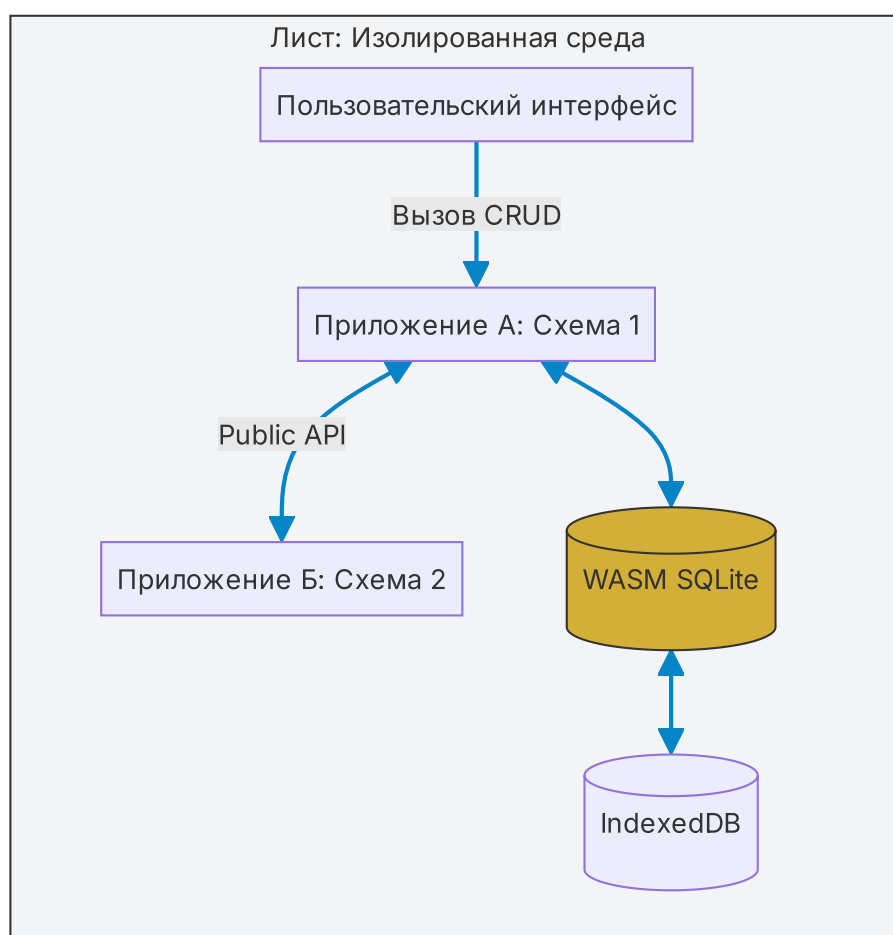
8. Интерфейс пользователя и прикладной уровень

7.1. Архитектура исполнения и изоляция

Каждый «Лист» представляет собой автономную изолированную среду (песочницу), внутри которой приложения работают без прямого доступа к сетевым ресурсам или API веб-платформы.

Слой данных: Активные хранилища обрабатываются в памяти через WASM SQLite, а постоянное хранилище обеспечивается IndexedDB.

Прикладной уровень: Приложения инкапсулируют свою логику в отдельных схемах и общаются друг с другом через строго определенные открытые API, что обеспечивает высокий уровень безопасности и стабильности.

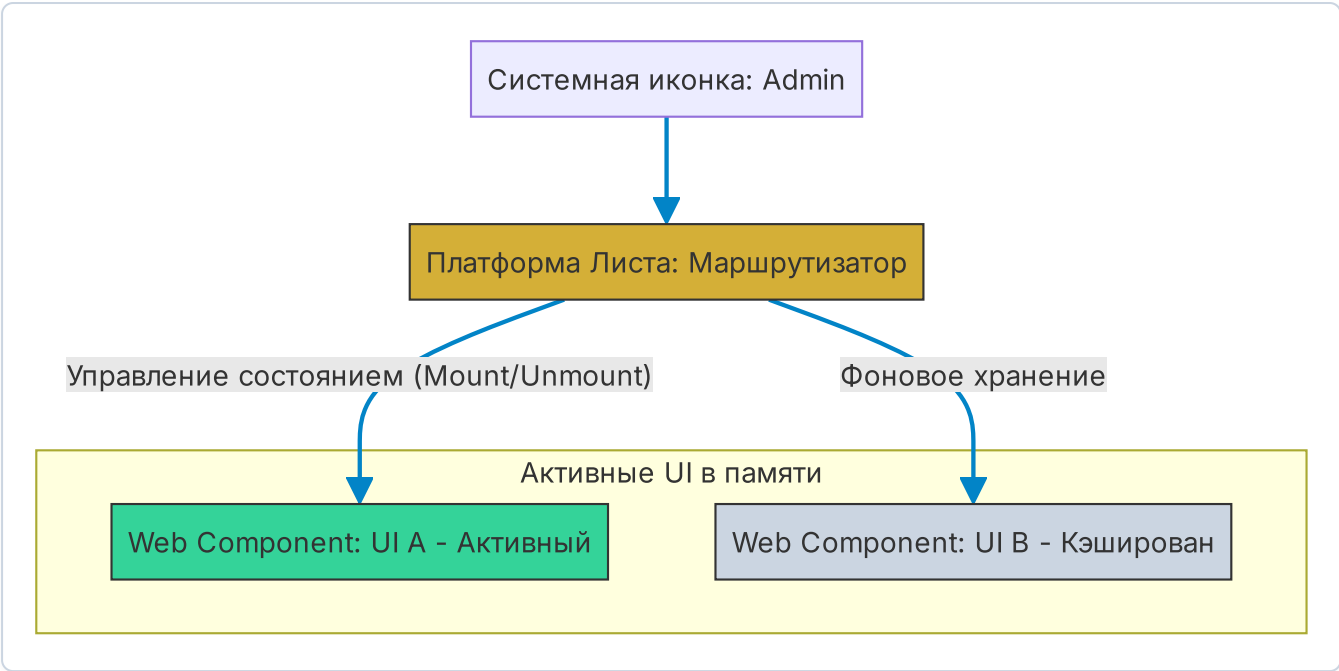


7.2. Композиция UI и управление состоянием

Интерфейсы пользователей реализованы как полноэкранные SvelteJS Web Components, которые управляются централизованным маршрутизатором платформы.

Жизненный цикл: Платформа «Листа» берет на себя управление монтированием и переходами (Routing API), обеспечивая мгновенный отклик за счет фонового кэширования неактивных интерфейсов.

Платформенное меню: Доступ к глобальным настройкам и навигации осуществляется через выделенный элемент управления («Icon Overlay»), скрытый от прямого доступа приложений.



9. Управление жизненным циклом и обновление системы

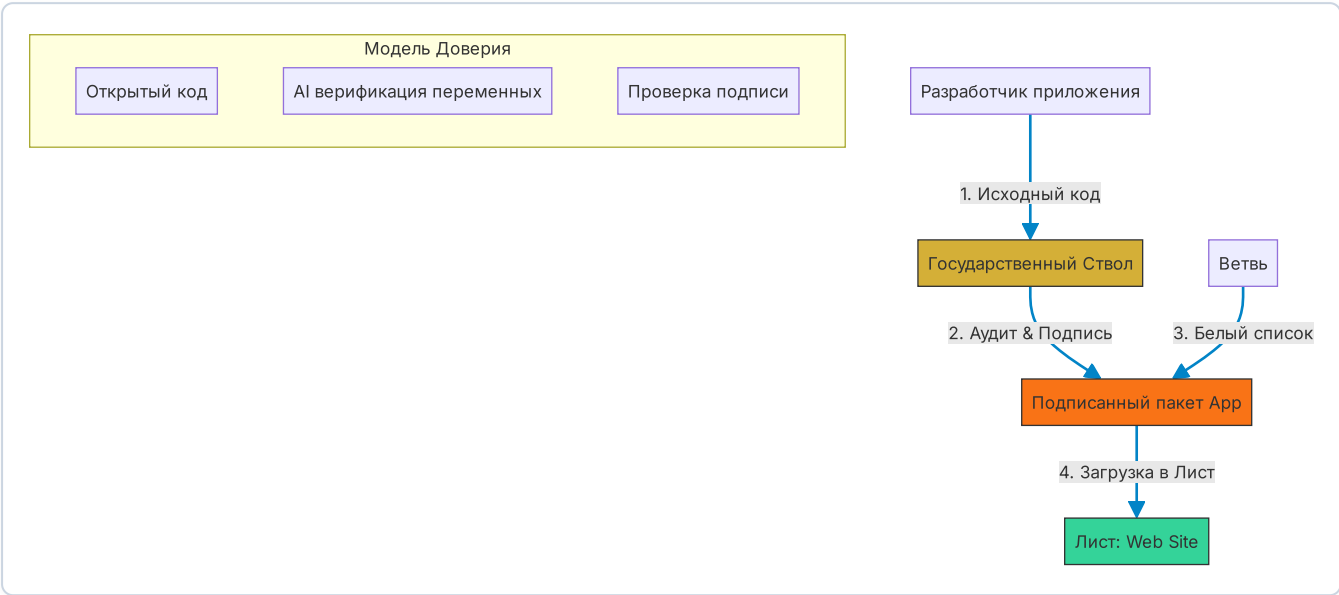
8.1. Иерархия доверия и верификация кода

Для обеспечения безопасности системы используется строгая модель доверия, исключающая установку несанкционированного ПО.

Двойная подпись: Каждое приложение (Web Component) и обновление платформы подписывается двумя сторонами: разработчиком и государственным «Стволом» (Trunk).

Белый список «Ветвей»: Только приложения, одобренные «Ветвью», доступны для загрузки и исполнения на «Листах».

Прозрачность: Исходный код является открытым, что позволяет проводить независимый аудит безопасности и использование AI-инструментов для анализа потоков данных.



Open Source & AI-Audit: Весь код открыт, что позволяет сторонним суверенным государствам и пользователям проверять отсутствие «закладок» и утечек данных.

Модель двойной подписи (Разработчик + Государственный Ствол) в сочетании с открытым исходным кодом создает прозрачную среду, где безопасность проверяется не «на слово», а через верифицируемый путь от исходного кода до исполняемого WASM-модуля.

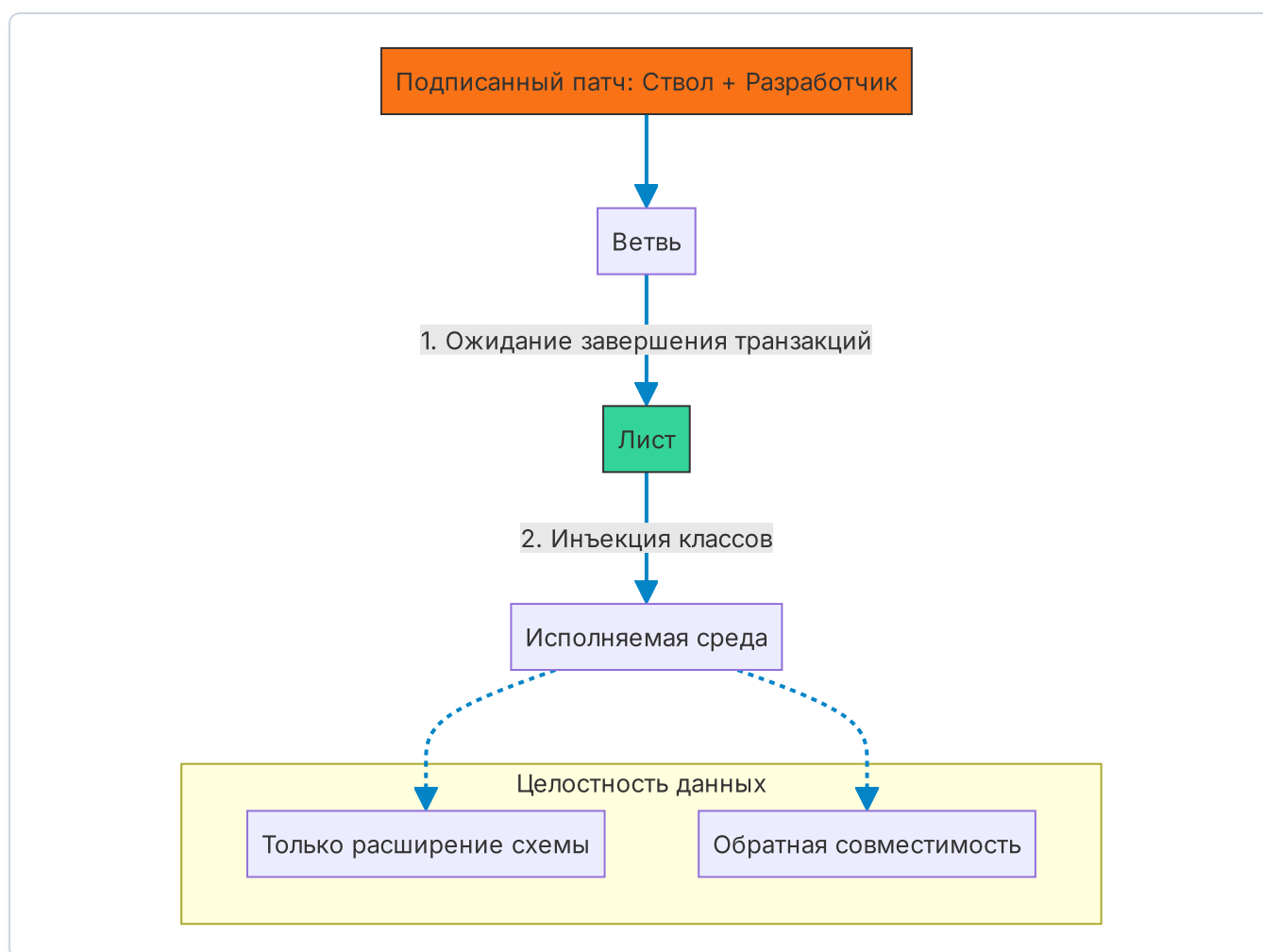
8.2. Инъекция патчей и эволюция схем данных

Система поддерживает непрерывное обновление без простоев и принудительных миграций данных.

Hot Patching (Инъекция классов): Обновление кода платформы и приложений происходит путем «горячей» подмены классов в рантайме (class injection). Платформа ожидает завершения активных транзакций, обеспечивая целостность состояния системы.

Аддитивная модель схем: Структура данных (таблицы и столбцы) может только расширяться. Это гарантирует 100% обратную совместимость, позволяя старым и новым версиям приложений работать с одними и теми же хранилищами.

Ответственность разработчика: При необходимости радикальных изменений структуры данных, логика миграции остается в зоне ответственности разработчика приложения, при этом система поощряет стабильность через экономические стимулы API.



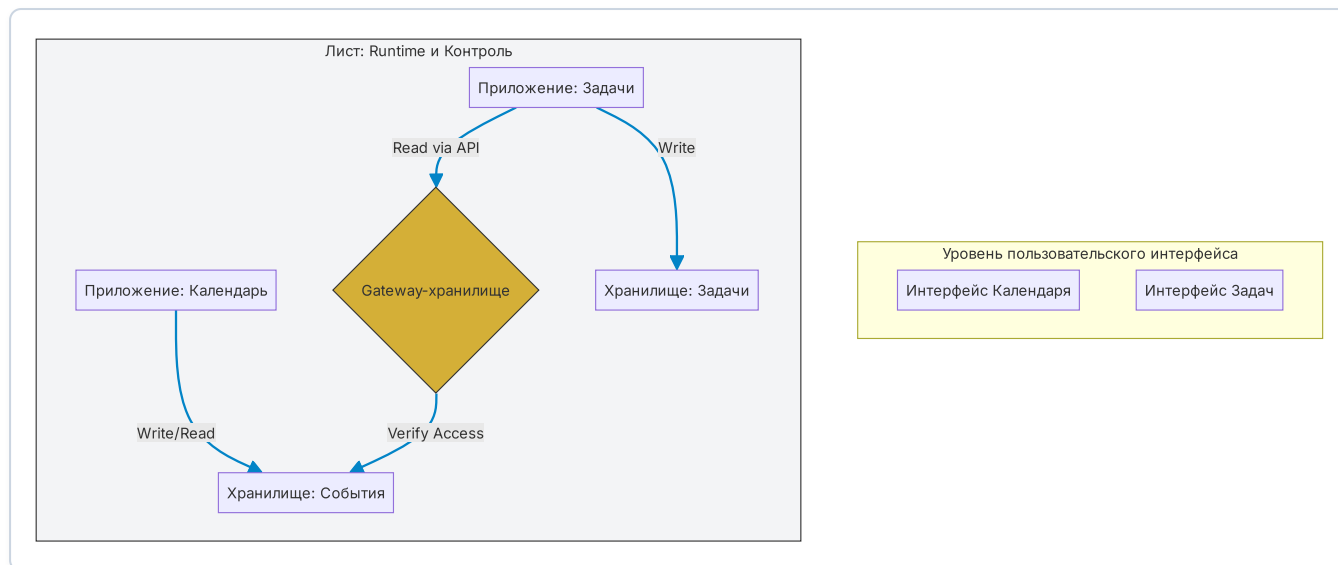
10. Кооперативные приложения и изоляция данных

13.1. Архитектура кооперативного доступа

В данной системе приложения не работают в вакууме. Для совместного использования данных они используют Gateway-хранилища, которые выступают в роли контролируемых точек входа. Безопасность обеспечивается тем, что любое межприложенческое взаимодействие проходит через API-вызовы, валидируемые на уровне «Листа» (Runtime).

Пример «Календарь + Задачи»: Приложение «Задачи» запрашивает доступ к данным «Календаря» для автоматического создания напоминаний.

Изоляция: Приложение «Задачи» не имеет прямого доступа к базе данных «Календаря». Оно обязано вызвать API-метод `ReadCalendarEvents()`, который проходит проверку Runtime-контроля.



Через свою настройку приложения могут открыть право прямого доступа к таблицам в своей схеме данных для создания запросов с соединением SQL (SQL join queries) но не могут дать право изменения. Все изменения данных в схеме приложения должны быть сделаны через API-методы этого приложения.

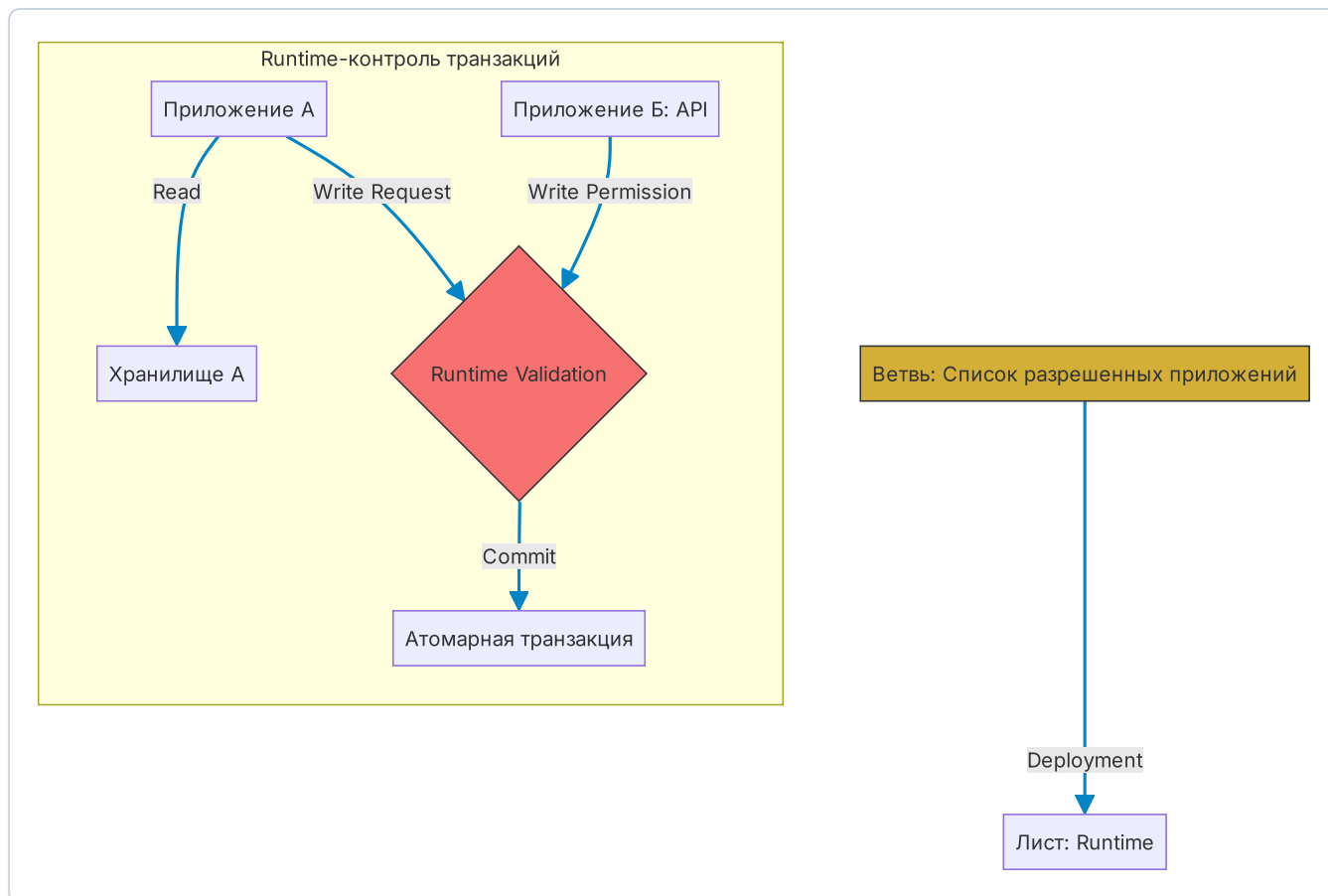
13.2. Обеспечение безопасности и целостности

Система предотвращает утечки данных и злоупотребления через многоуровневый аудит:

Deployment Lockdown (Уровень Ветви): Установка приложения на «Лист» возможна только из доверенного списка Ветви. Приложение, не прошедшее верификацию, физически не загружается и не имеет доступа к API.

Runtime-валидация (Уровень Листа): При совершении транзакции записи, Лист проверяет, было ли получено право на запись в рамках текущей сессии UI. Это предотвращает сценарий, когда «злонамеренный» UI пытается подменить данные, используя API другого приложения.

Анализ исходного кода: Поскольку все приложения — Open Source, система использует AI-ревью для отслеживания переменных и потоков данных (data flow analysis), что позволяет автоматически помечать «подозрительные» приложения как нежелательные для определенных типов хранилищ.



13.3. Применение в кооперативных сценариях

Приложения могут вызывать API других приложений для чтения или читать напрямую, но обязаны использовать API для записи. Это гарантирует, что любое изменение данных фиксируется как «транзакция приложения», которая проходит проверку на легитимность.

Примечание: Если приложение пытается совершить запись в хранилище, к которому оно не имеет явного права доступа в текущем контексте (session context), транзакция блокируется на уровне ядра Листа, обеспечивая полную изоляцию данных даже при активной кооперации приложений.

Часть IV: Идентичность и Федерация

11. Идентичность пользователя, Keyring и портативность

11.1. Архитектура управления доступом (Keyring System)

Система идентичности строится на принципе сегментированного владения правами. Пользователь не ограничен единым ключом, а управляет множеством «Keyring-хранилищ», каждый из которых привязан к конкретной Ветви (личной, государственной или бизнес-ветви).

Сегментация: Разделение на «Рабочий», «Личный» и другие Keyring-и позволяет физически разграничить данные разного уровня секретности.

Индексная модель: Индексное хранилище Keyring является «связующим звеном», объединяющим все миры пользователя в единую логическую систему без необходимости физического смешивания данных.

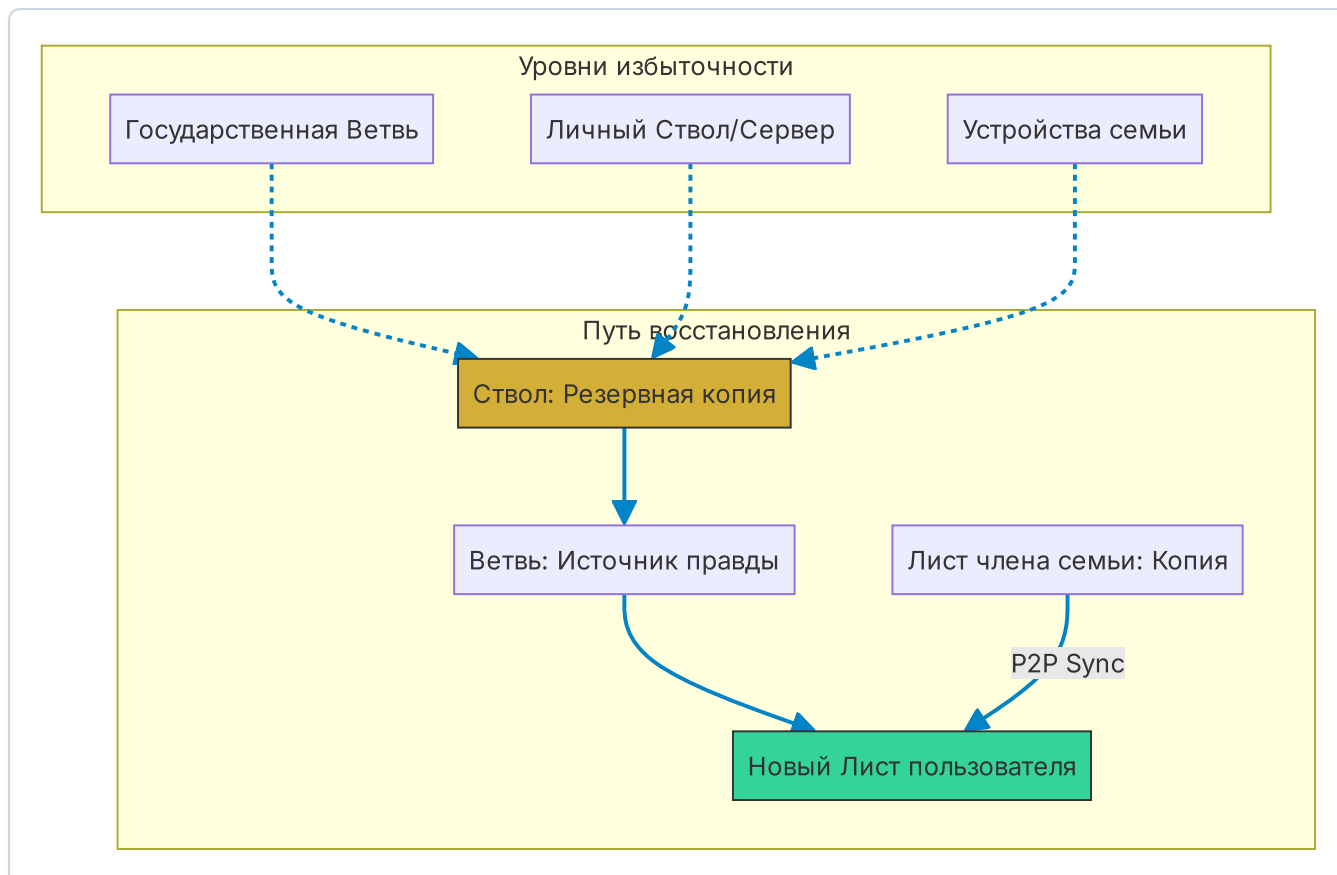


11.2. Резистентность и восстановление

Поскольку данные и транзакционные логи хранятся не на клиентском устройстве («Листе»), а реплицируются вверх по иерархии (вплоть до Ствола), система обеспечивает высокую живучесть данных.

Автономное восстановление: При потере физического устройства (Листа), пользователь инициализирует новый Лист и восстанавливает свое состояние путем синхронизации с Ветвью/Стволом.

Избыточность: Индексное хранилище Keyring копируется на уровне Стволов и может быть дублирован на доверенных Листах (например, устройствах семьи или доверенных лиц), что исключает риск безвозвратной потери ключей.



Fork & Lock: В корпоративной среде, при прекращении отношений с сотрудником, организация использует механизм «форка» (копирования) хранилищ для блокировки доступа пользователя и изъятия прав владения корпоративными данными.

12. Межгосударственное взаимодействие и федерация суверенных юрисдикций

10.1. Иерархический протокол соединения (Cross-Sovereign Link)

Взаимодействие между «Листами» разных юрисдикций организовано через иерархическую структуру, обеспечивающую суверенный контроль над входящим и исходящим трафиком.

Маршрутизация: Все запросы на соединение проходят через цепочку «Лист → Ветвь → Ствол». Это позволяет суверенным агентствам контролировать, с какими регионами или конкретными «Листами» разрешен обмен данными.

Черный ящик (Black Box): По умолчанию, идентификация «Листа» из другой страны ограничена идентификатором и страной происхождения, что обеспечивает баланс между прозрачностью и безопасностью.

Плоскость данных: После успешного установления доверительного соединения (Handshake) через «Стволы», передача данных между «Листами» осуществляется напрямую по протоколу WebRTC для минимизации задержек.



Syntax error in text

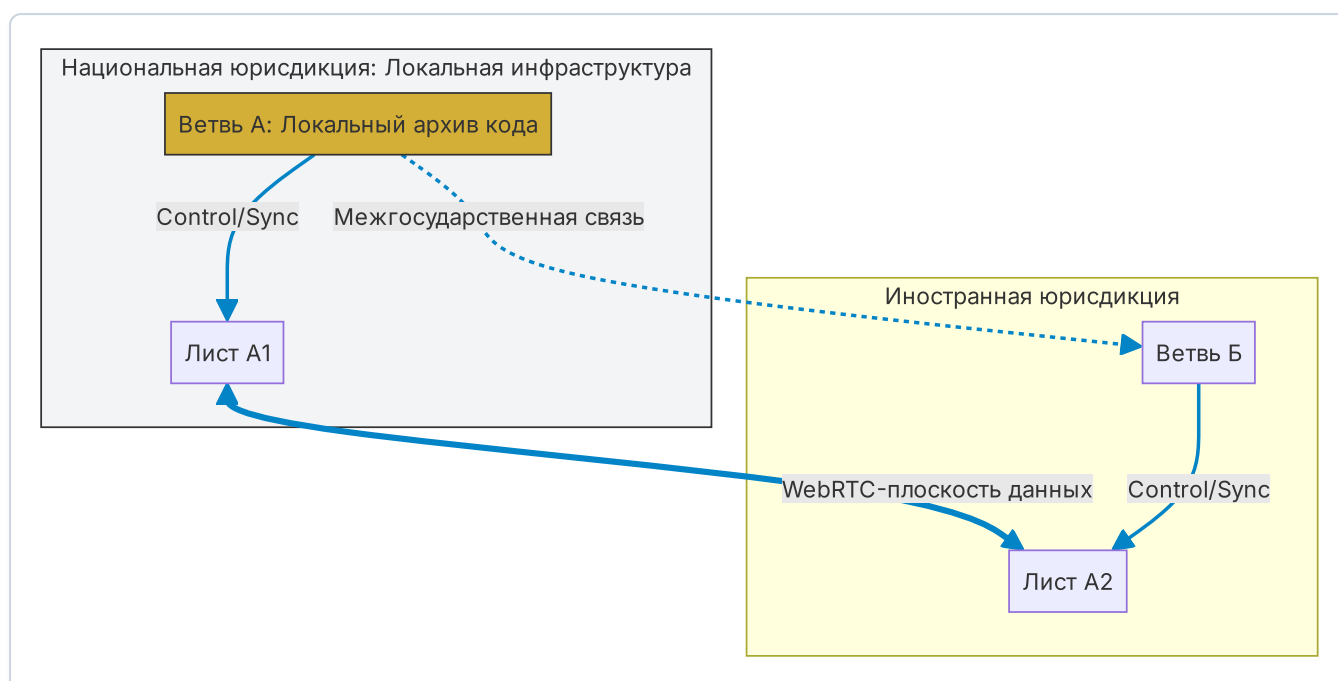
mermaid version 10.9.8

10.2. Резистентность и локальная автономность

Архитектура обеспечивает живучесть приложений даже при возникновении «геополитического разрыва» (прерывании связи между «Стволами»).

Локальный контроль: Каждая «Ветвь» физически владеет копиями исходного кода установленных приложений. Система не зависит от доступности «Ствола» страны-разработчика для функционирования уже установленного ПО.

Автономная работа: При потере связи с внешними «Стволами», локальные «Листы» продолжают функционировать, используя кэшированные данные и локально установленные приложения.



Иерархия коммуникации: * плоскость управления: (Логи, транзакции, регистрация) — всегда через «Ветви»/«Стволы».

Плоскость данных: (Peer-to-Peer) — WebRTC напрямую между «Листами» (минимизирует нагрузку при работе внутри одной страны).

10.3. Экономика трансграничных операций

Локализация валюты: Пользователь всегда платит в валюте своей страны (согласно регистрации его Листа).

Абстракция разработчика: Приложение получает доход в валюте «Листа» пользователя, а вопросы конвертации токенов между юрисдикциями остаются на усмотрение самих поставщиков приложений.

Контроль доступа: Суверенные государства могут гибко настраивать «прозрачность» своих ветвей (от Black Box до открытого распределенного индекса), что позволяет доверять данным иностранных приложений, не раскрывая внутреннюю топологию сети.

Часть V: Экономика Платформы

13. Система экономики API

9.1. Разделение прибыли

Платформа обеспечивается прибылью от рекламы и подписок. Прибыль делится между арендаторами мощностей, разработчиками ПО и пользователем, в случае просмотра рекламы. Прибыль полученная пользователем частично обслуживает хранение его зашифрованных хранилищ на общественной ветке и частично накапливается и может быть использована в качестве разработчикам ПО оплаты за премиальные функции приложений.

9.2. Ликвидность

Платформа может использовать суверенную цифровую валюту государства на прямую или использовать систему жетонов или токенов (). В случае использования жетонов валюты арендаторы и разработчики смогут обменивать их через финансовую систему государства. Ликвидность жетонов полученных пользователями за просмотр рекламы в сфере компетенции суверенных государств.

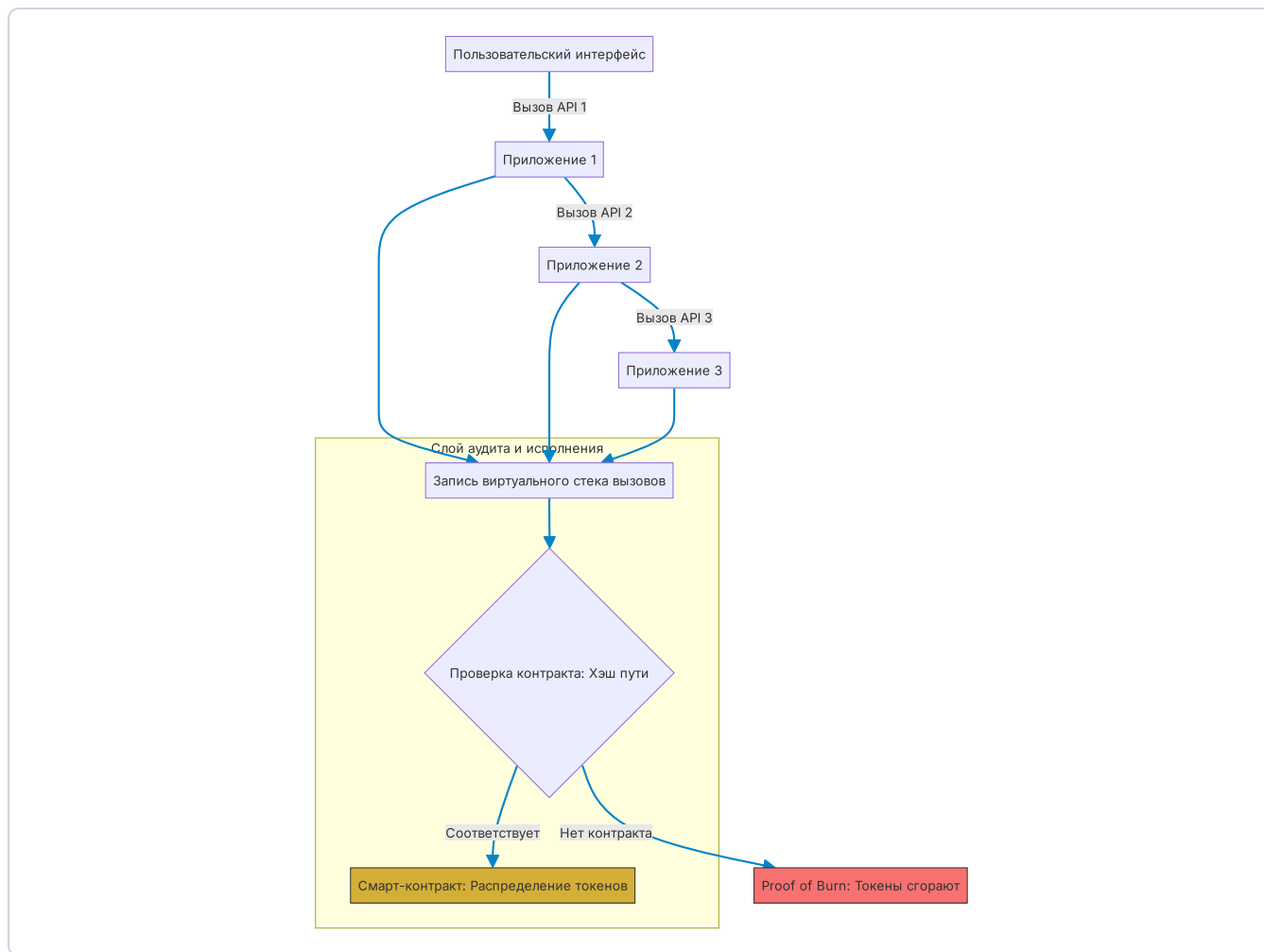
9.3. Технический аудит и запись API-вызовов

Платформа обеспечивает неизменяемый учет вызовов через интеграцию платформенного кода и государственного блокчейна жетонов или национальной цифровой валюты.

Виртуальный стек вызовов: Платформа автоматически записывает путь API-вызовов для каждой операции рендеринга страницы.

Смарт-контракт: Записанный хэш пути вызовов сравнивается с общественно согласованным контрактом (через GitHub PR), что определяет автоматическое распределение вознаграждений.

Прозрачность: Данные записываются в блокчейн анонимно, без привязки к профилю пользователя, сохраняя приватность.



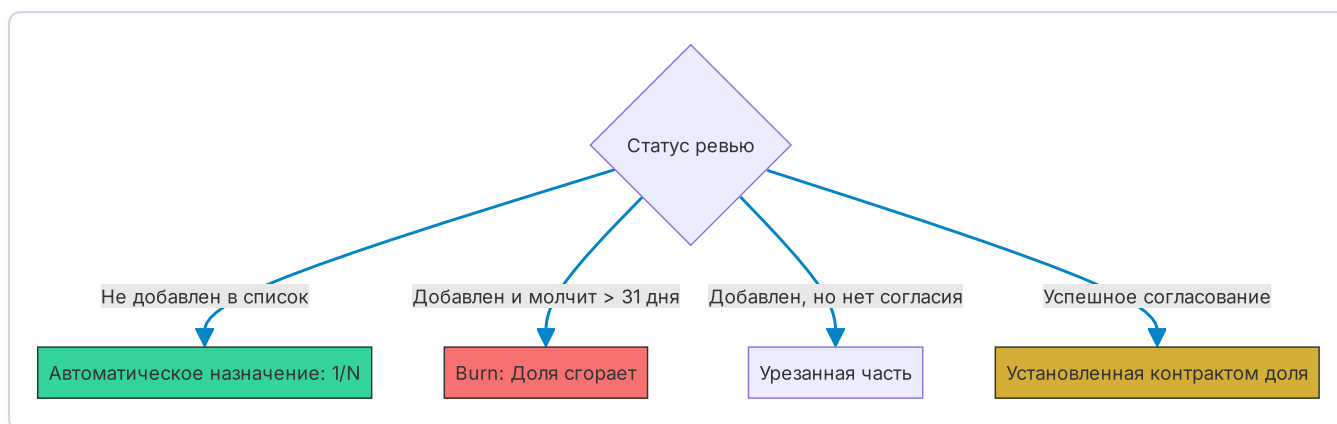
9.4. Алгоритм разрешения споров и распределения долей

Система автоматически вычисляет долю, основываясь на статусе участия разработчика в процессе согласования:

Автоматическое включение: Если разработчик API не был добавлен в ревью, он автоматически получает арифметическую среднюю долю.

Игнорирование (молчание): Если разработчик не ответил в течение 31 дня (конфигурируемо), его доля сгорает.

Недостижение согласия: Если стороны вели переговоры, но не сошлись в цене, участнику полагается урезанная часть от его «средней арифметической» доли.



9.5. Механизм разрешения конфликтов (Game Theory)

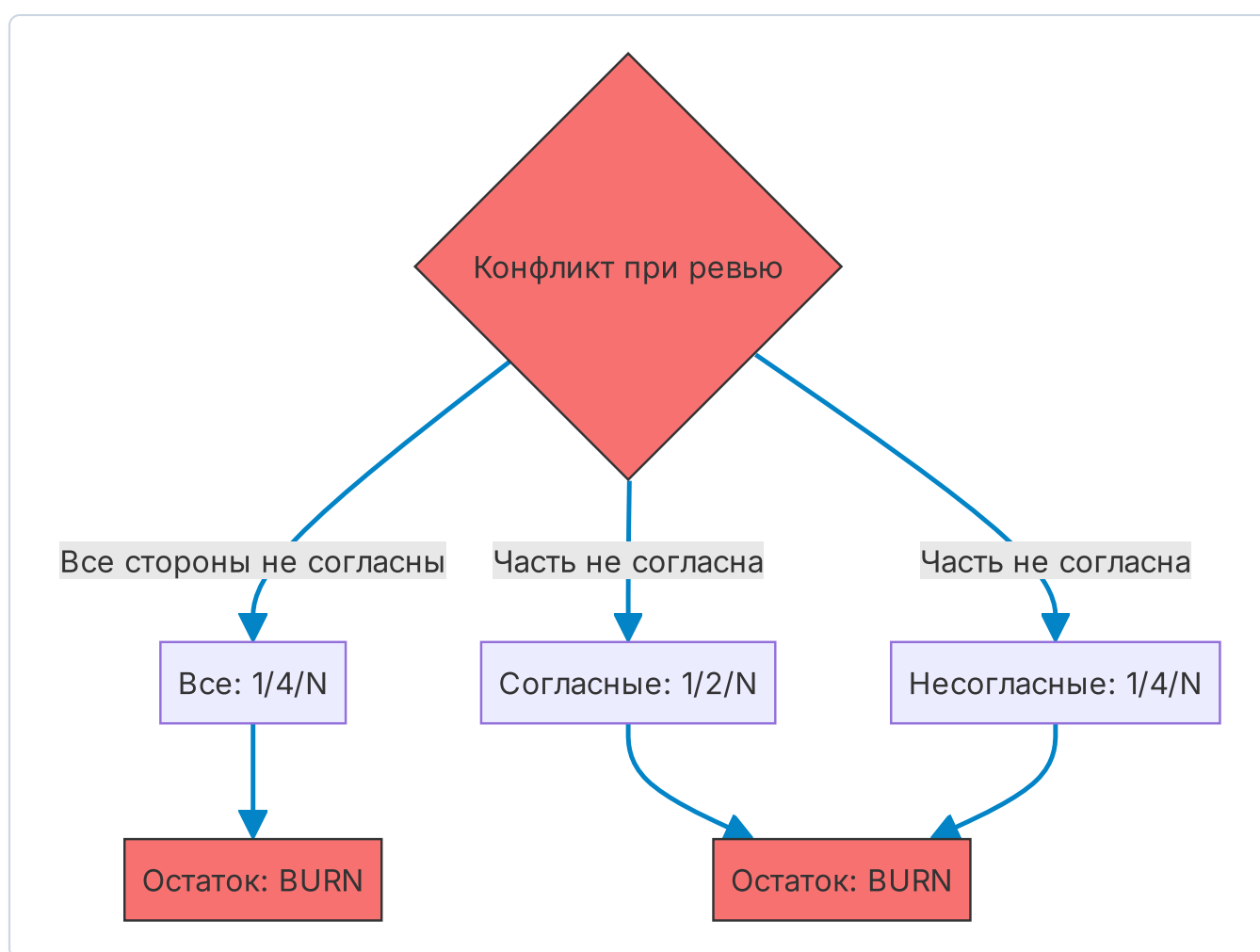
Для обеспечения консенсуса между разработчиками, система применяет жесткие штрафные санкции при отсутствии соглашения о распределении доходов.

Компромиссная модель: Использование штрафов для принуждения к переговорам.

Матрица распределения долей:

Участник	При достижении согласия	При отсутствии согласия
Сторона, согласившаяся на компромисс	Полная доля (по умолчанию $1/N$)	$1/4/N$
Сторона, отказавшаяся от компромисса	—	$1/4$ доли
Сторона отказавшаяся договариваться	0	0

Если стороны не достигают консенсуса, система применяет «агрессивное сокращение» долей, чтобы принудить участников к диалогу.



Этот механизм защищает систему от «захвата» прибыли (rent-seeking) и заставляет участников быть конструктивными.

9.6. Управление монетарной политикой

Смарт-контракты оперируют токенами UTXO или государственной цифровой валютой. Механизм «сжигания» (Burn) не является окончательным уничтожением активов:

Государственный контроль: Регулирующее Агентство управляет кошельками, куда поступают «сожженные» токены.

Гибкость: Агентство может принимать решение об окончательной дефляции или перенаправлении средств на субсидирование критической инфраструктуры, обеспечивая ликвидность экономики платформы.

9.7. Экономика API и мотивация сохранения структуры схем

Разработчики финансово мотивированы сохранять обратную совместимость и неизменность структуры таблиц (для поддержки внешних ссылок, Foreign Keys и публичных API-вызовов), что естественным образом ограничивает хаотичные изменения схем. Каждая версия схемы регистрируется в глобальном реестре Ствола, а вызовы методов модулей монетизируются через автоматическое распределение роялти по стеку вызовов.

9.8. Архитектура минимальных смарт-контрактов: Конечный автомат

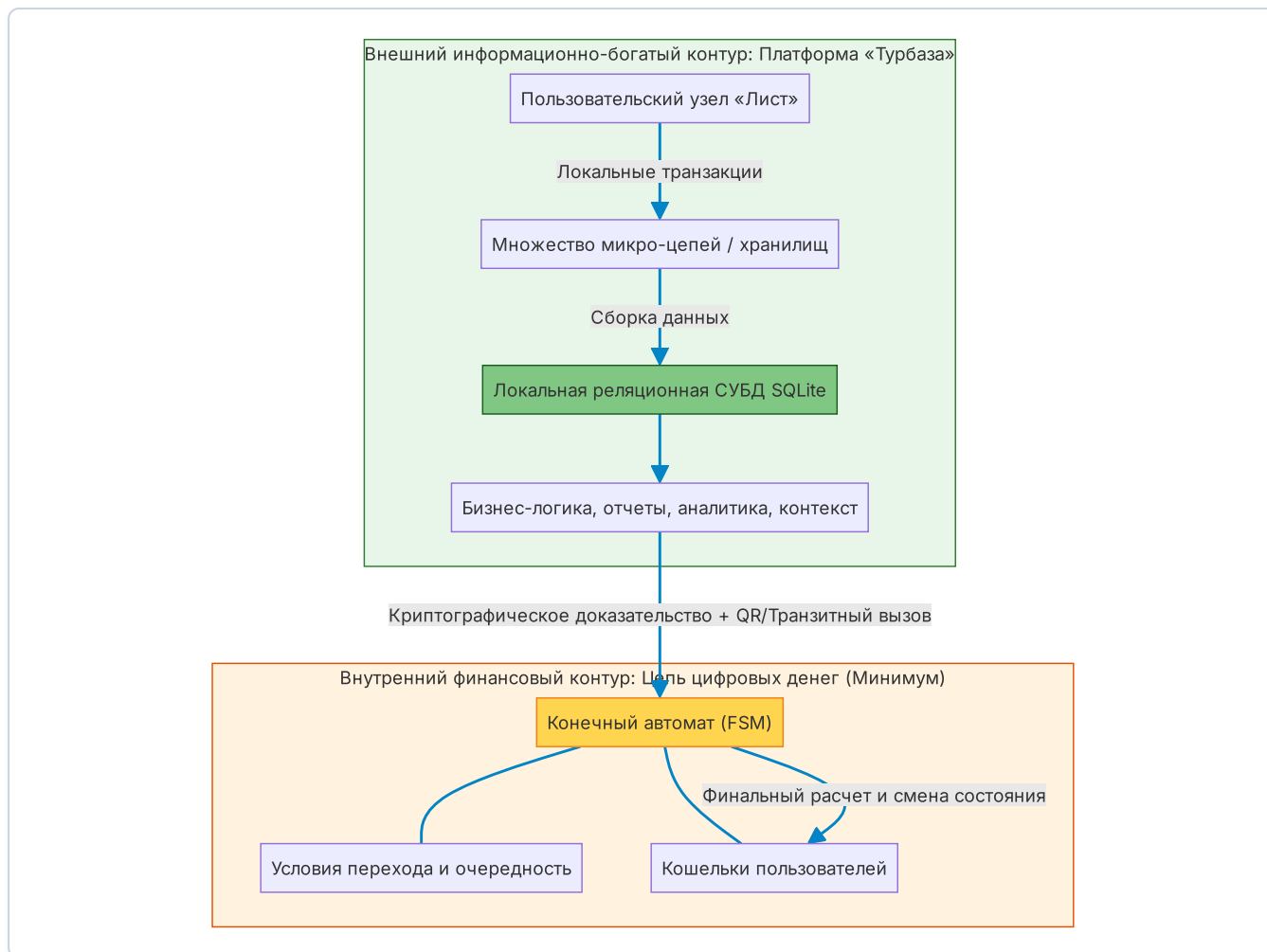
В платформе «Турбаза» концепция смарт-контрактов кардинально очищена от избыточных вычислений и ресурсоемкой бизнес-логики, перегружающих традиционные распределенные реестры.

Математическая сущность:

В своей основе смарт-контракт представляет собой минимальный детерминированный **конечный автомат** с набором условий перехода:

- **Соблюдение условий и очередности:** Платформа гарантирует, что условия выполняются строго в заданном протоколом порядке.
- **Верификация полномочий участников:** Переход состояния происходит только при предъявлении валидных криптографических доказательств (ЭЦП ГОСТ / Ed25519) авторизованными сторонами.
- **Изоляция встроенной логики кошельков:** Единственным критическим ресурсом, требующим платформенной логики непосредственно внутри цепи цифровых денег (например, Цифрового рубля Банка России или УТХО-реестра), являются кошельки пользователей.

Все остальные функции — бизнес-процессы, документооборот, учет номенклатуры, сложная аналитика — полностью выносятся во внешний контур.



9.9. Внешний информационно-богатый контур: Микро-цепи и локальный SQL

Платформа «Турбаза» идеально дополняет финансовые смарт-контракты, создавая внешний информационно-богатый контур:

- **Агрегация микро-цепей в локальный SQL:** На пользовательском аппарате («Лист») данные из множества независимых микро-цепей (хранилищ) собираются в единую стандартную реляционную базу данных SQLite.
- **Универсальная бизнес-логика:** Разработчики описывают прикладную логику с использованием простых, высокопроизводительных реляционных SQL-запросов (`SELECT` , `JOIN` , оконные функции), не прибегая к дорогостоящим вычислениям в блокчейне.
- **Синхронизация дельт:** Пользовательские модификации фиксируются в локальной БД и попутно распределяются в виде криптографических блоков изменений (транзакций) по соответствующим микро-цепям.
- **Связка с владением кошельком:** В сочетании с криптографическим доказательством владения кошельком на аппарате пользователя это позволяет дополнить базовую логику смарт-контракта богатым и сложным жизненным контекстом реального мира.

9.10. Канонические практические сценарии

Минималистичные смарт-контракты «Турбазы» строятся из стандартных модульных примитивов:

- Временные функции: `onceIn(seconds, ...)`

- Логические операторы: `lte(a, b)`, `gte(a, b)`, `or(a, b)`, `and(a, b)`
- Финансовые примитивы: `set(amount, address)`, `add(amount, address, target)`, `share(ratio1, address1, ratio2, address2, ...)`

Сценарий 1. Целевая оплата школьного обеда (Ограничения, конфиденциальность и кэширование)

Родитель формирует смарт-контракт целевых школьных расходов:

- Периодичность: один раз в день (`onceIn(86400, ...)`)
- Лимит: не более 300 рублей (`lte(300, ...)`)
- Целевое назначение: только на завтрак или обед (`or(A, B)`)

Формула смарт-контракта:

```
onceIn(86400, lte(300, or(A, B)))
```

Где `A` и `B` — обезличенные стандартные адреса-заполнители, связанные с их реальным назначением в микро-цепях «Турбазы» (`A` — Завтрак, `B` — Обед).

Преимущества архитектуры:

- **Максимальная конфиденциальность:** Использование стандартных адресов `A` и `B` полностью вуалирует реальную суть контракта в публичной цепи. Внешний наблюдатель видит лишь вызов абстрактных функций.
- **Кэширование контрактов:** Типовой контракт хешируется при создании и повторно используется миллионами родителей и школьников без повторного развертывания уникального байт-кода.
- **Оптимизация нагрузки:** Вся детализация чеков (что именно съел ученик, состав блюд, КБЖУ, точное время) хранится в микро-цепях «Турбазы» на смартфонах родителей и школьных локальных серверах. Платформа смарт-контрактов обслуживает исключительно перевод денежных средств, снижая транзакционные издержки практически до нуля.

Сценарий 2. Логистический эскроу доставки еды (Синхронизация в микро-цепи заказа)

Заказ создается в выделенной микро-цепи (хранилище) и реплицируется между узлами участников: рестораном, курьером и клиентом.

- Хранилище заказа создается на телефоне покупателя и через районную Ветку передается:
- Начальнику смены ресторана `A` (маркирует заказ как приготовленный);
- Курьеру компании `B` (маркирует приемку посылки и доставку по адресу);
- Покупателю (наблюдает прогресс и подтверждает получение).
- Смарт-контракт в финансовом контуре предельно компактен:

```
sum = set(400, A)
add(100, B, sum)
```

Вся операционная координация и трекинг происходят в локальном хранилище «Турбазы». Смарт-контракт лишь верифицирует финальную сумму и исполняет атомарный перевод при наличии подписей всех трех сторон.

Сценарий 3. Рекурсивное распределение дохода (Revenue Sharing в экосистеме приложений)

В «Турбазе» приложения свободно комбинируют открытые схемы данных и программные оболочки других разработчиков. Доход от показа контекстной рекламы или подписок автоматически расщепляется:

- Если приложение разработчика **Б** использует библиотеку разработчика **А**, доход делится формулой:

$$B = \text{share}(0.5, B, 0.5, A)$$

- Композиции контрактов вкладываются друг в друга без ограничений глубины:

$$E = \text{share}(0.2, D, 0.6, B, 0.2, G)$$

- Сверх того, часть дохода от просмотренной рекламы может быть поделена с пользователем, который смотрел эту рекламу:

$$\text{share}(0.1, Я, 0.9, E)$$

- Детерминированное вознаграждение операторов периферийных мощностей (Capacity Renters / Ветка):
Для компенсации расходов операторов районных Ветвей на секвенирование логов, поддержание Trie-индексов и ретрансляцию трафика контракт включает прямое алгоритмическое расщепление в пользу узла Ветка:

$$\text{share}(0.05, \text{Ветка}, 0.1, Я, 0.85, E)$$

Это гарантирует экономическую самодостаточность децентрализованной периферийной инфраструктуры: операторы узлов получают прямое вознаграждение в Цифровом рубле за предоставление вычислительных и дисковых ресурсов без посредников и рекламных монополий.

9.11. Контрактные оболочки: Приватность кошельков и управление устройствами

Ключевым требованием платформы «Турбаза» является соблюдение государственной тайны и приватности граждан (152-ФЗ).

Защита номеров кошельков

Прямая публикация номеров криптографических кошельков в открытых контрактах недопустима, так как позволяет сопоставлять финансовые потоки пользователей и бизнеса. В «Турбазе» реальные номера кошельков изолируются внутри **базовых контрактных оболочек**. В вышестоящие бизнес-контракты включаются исключительно адреса оболочек. *Результат:* Каждый участник сделки знает адрес собственного кошелька, но взаимодействует с контрагентами через непрозрачные криптографические шлюзы оболочек.

Сменяемые целевые идентификаторы

Все расчетные операции выполняются через прикрепленные оболочки смарт-контрактов:

- Устойчивость при потере смартфона:** К пользовательскому устройству привязываются одна или несколько оболочек-прокси. В случае утери или кражи смартфона привязанные оболочки мгновенно отзываются и аннулируются. На новом устройстве выпускаются новые оболочки без необходимости проведения рискованных и сложных криптографических процедур смены мастер-ключа или корневого номера кошелька.
- Целевые ограничения:** Оболочки смарт-контрактов наделяются встроенными правилами (лимиты суточных трат, разрешенные категории расходов, географические зоны).

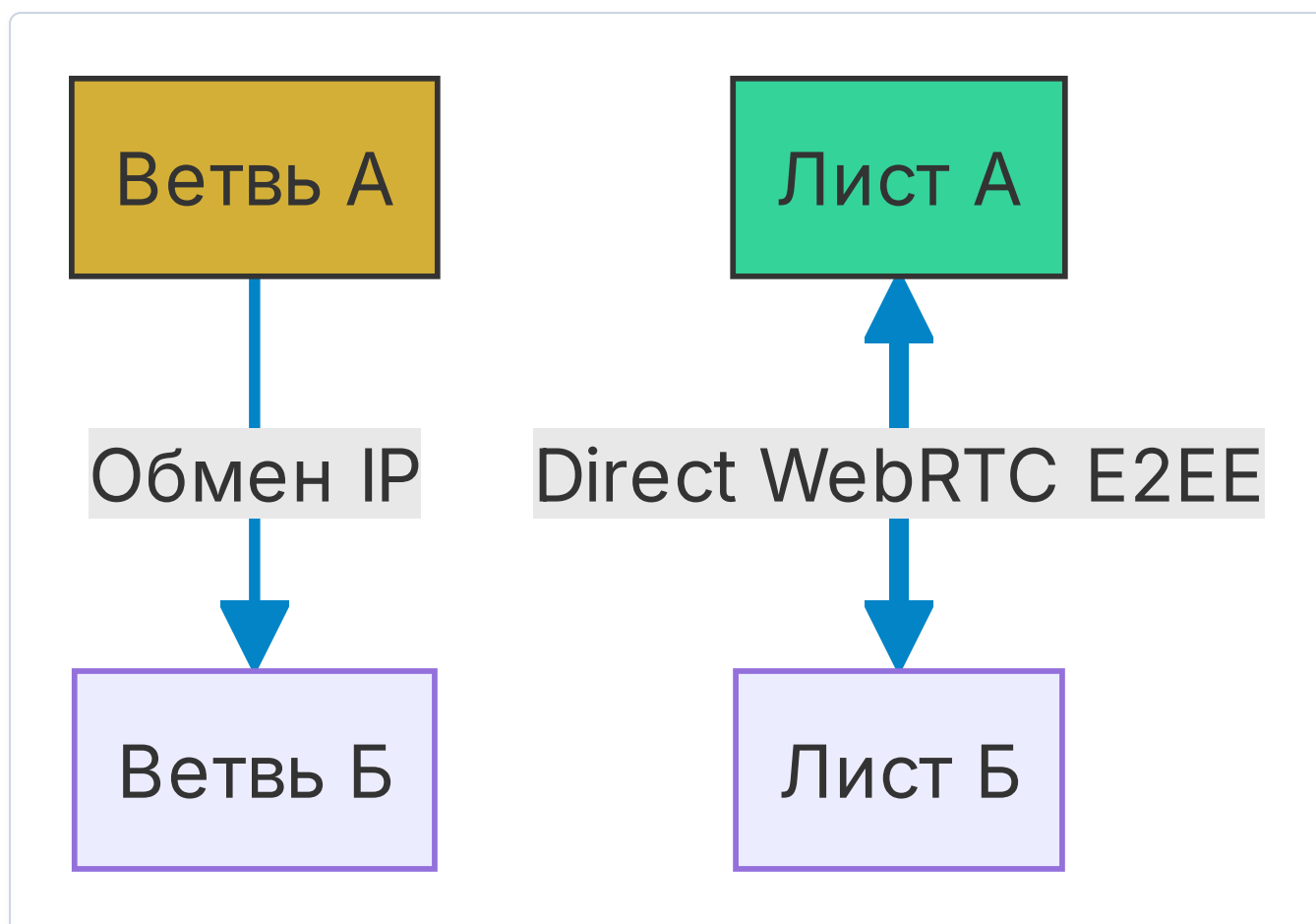
- **Общественные идентификаторы:** Для легитимного взаимодействия с государственными и корпоративными системами оболочки могут снабжаться открытыми верифицированными идентификаторами участников без деанонимизации базового банковского счета.

Часть VI: Прикладные примеры

14. Приватный чат и доверенная инициация соединений

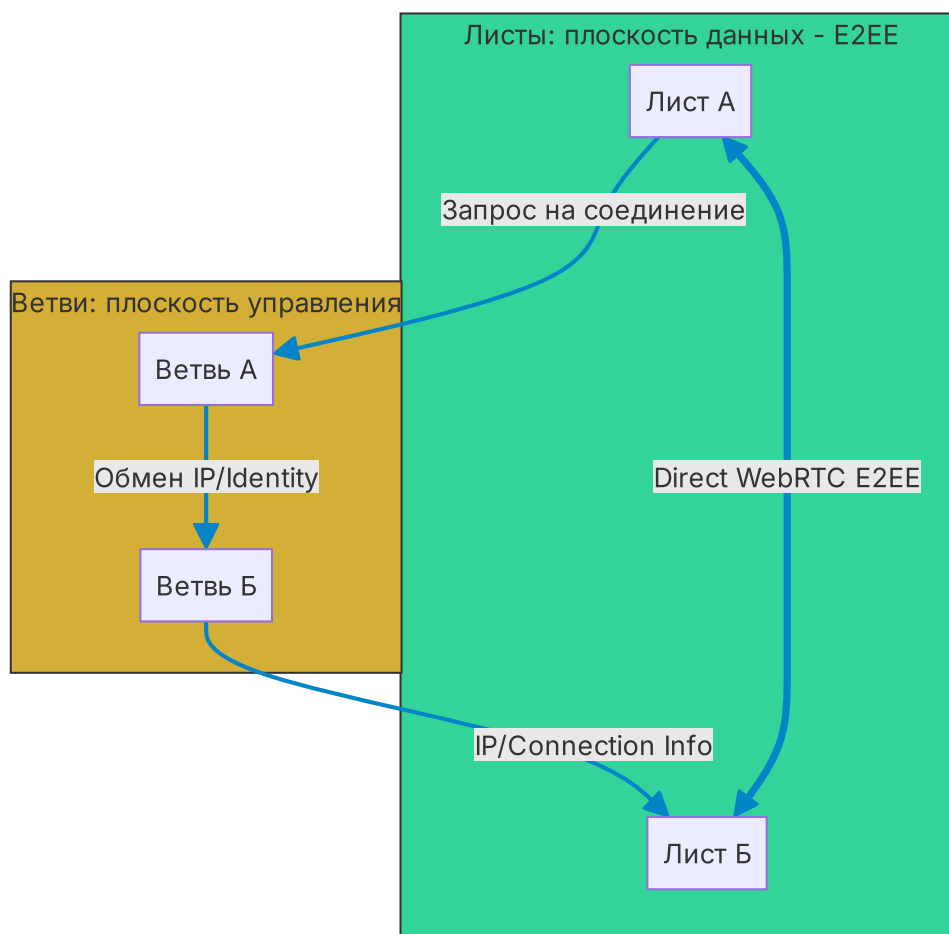
14.1. Разделение плоскости управления и плоскости данных

Для обеспечения суверенитета при сохранении приватности, система разделяет функции идентификации и обмена данными. Государственные или корпоративные Ветви выступают в роли «брокеров идентификации» (Identity Broker), в то время как само общение происходит напрямую между пользователями (P2P).



Инициация (плоскость управления): Ветви обмениваются IP-адресами и информацией об идентификации Листов, чтобы установить доверенное соединение.

Данные (плоскость данных): После установления соединения Листы общаются напрямую через протокол WebRTC, используя сквозное шифрование (E2EE), которое не проходит через серверы Ветви.

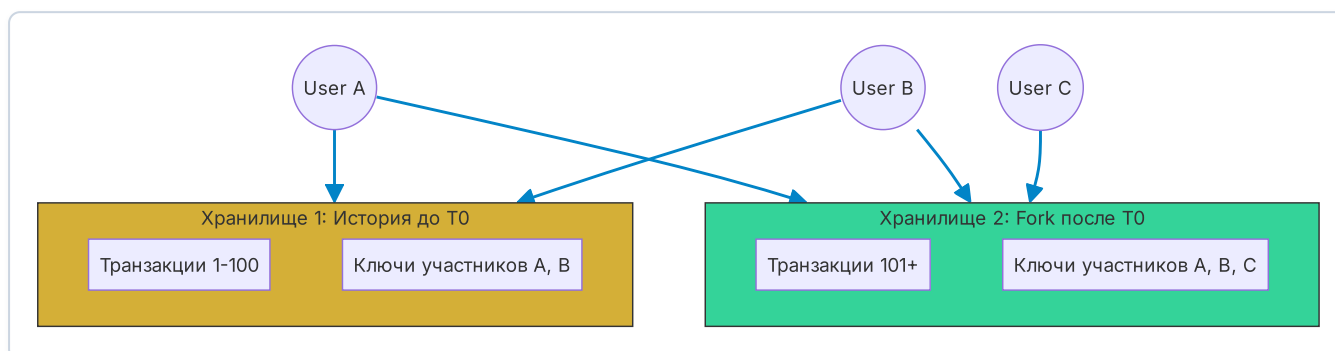


14.2. Управление историей и составом участников (Fork-механизм)

Для обеспечения безопасности групповых чатов система использует механизм «Форка» хранилища. Это позволяет добавлять новых участников в беседу без необходимости повторного шифрования исторического контента, доступ к которому должен быть ограничен.

Изоляция прошлого: Старое хранилище остается доступным только первоначальным участникам с их ключами.

Динамика состава: Новое хранилище создается («форкается») в момент добавления участника С, содержит новые ключи и доступен всем участникам (А, В и С).



Преимущества такого подхода:

Гарантированная изоляция: Новый участник физически не может получить доступ к данным до момента его добавления, так как у него нет доступа к ключам старого хранилища.

Целостность истории: Старая переписка остается «замороженной» в исходном хранилище, защищенной от несанкционированного чтения.

Упрощенная криптография: Нет нужды в ротации ключей или перешифровании огромных объемов данных — просто создается новая ветка с новым доверенным кругом лиц.

14.3. Сохранение данных

Пользователи имеют полное управление над тем, что сохранять из сессии WebRTC в их хранилище:

Шифрование данных в их хранилище использует ключи, которыми владеют только участники беседы, что обеспечивает приватность даже после завершения разговора.

Бинарные данные: Видео или аудио потоки могут быть зашифрованы ключом хранилища и сохранены для последующего воспроизведения.

Транскрипты: Текстовые логи бесед сохраняются как структурированные транзакционные записи, обеспечивая неизменяемость и возможность аудита внутри круга доверенных лиц.

15. Локализованные социальные сети и многоуровневое доверие

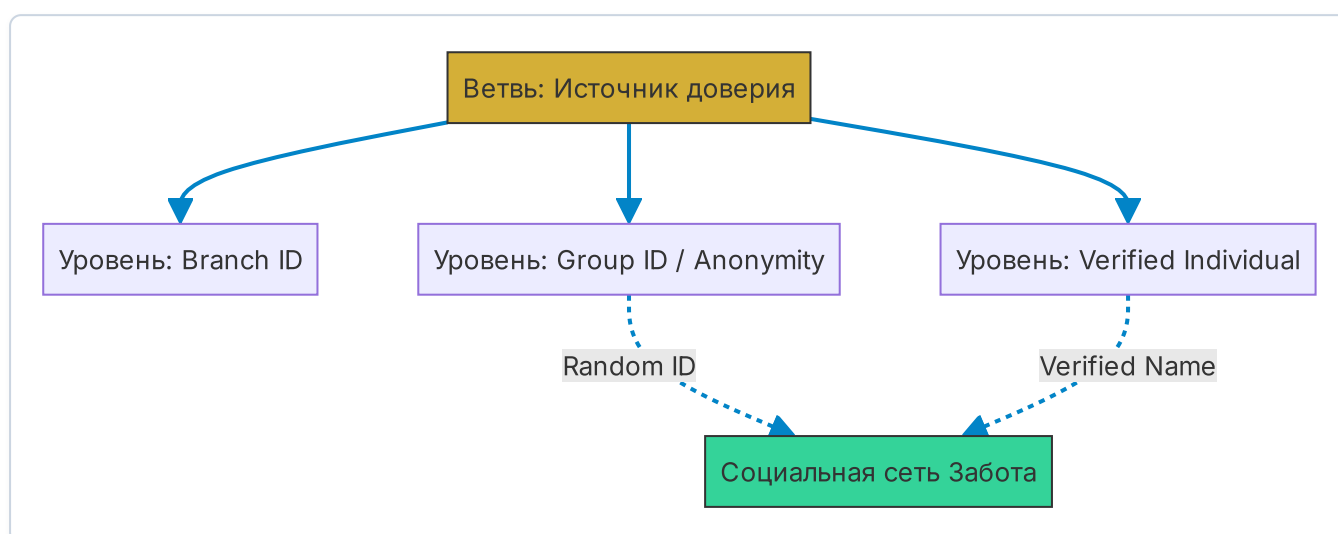
15.1. Трехуровневая модель верификации личности

Для создания доверенной среды система «Забота» использует режимы регистрации, где уровень доверия определяется способом регистрации пользователя через Ветви (Branches).

Полная анонимность (Branch ID): никто не знает пользователя, известен только район.

Социальная анонимность (Group Anonymity): регистрация через ID группы, где паспортные данные проверяются «на входе», но пользователь остаётся неизвестным в общественном контуре, получая случайный номер; экономически он известен Банку России (один кошелек на человека).

Поимённая регистрация (Individual Verified): полная верификация личности, предоставляющая максимальный уровень доверия для экономической деятельности; имя автоматически становится частью общественного контура.



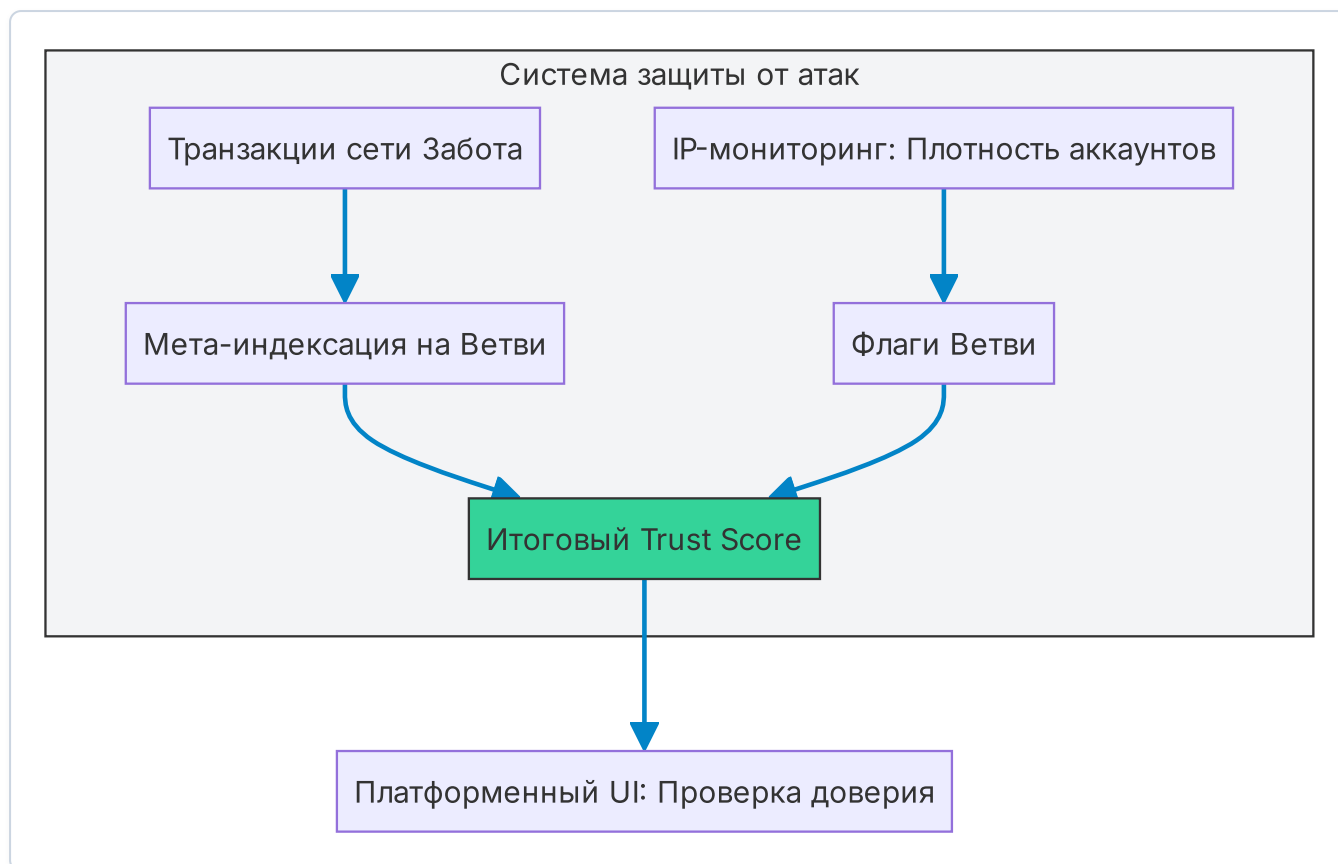
15.2. Защита от атак «Сибилла» (Anti-Sybil Architecture)

Для обеспечения честности репутационной системы «Забота» применяет статистический анализ метаданных, который выявляет попытки искусственного завышения рейтинга.

Сетевой антифрод: Ветви отслеживают плотность аккаунтов на один IP-адрес. Подозрительная активность (большое количество аккаунтов с одного узла) помечается флагами.

Мета-индексация: Все действия пользователей индексируются в хранилищах Ветвей, что позволяет вычислять доверие на основе истории деятельности и взаимных подтверждений (Cross-Verification). Репутация не просто «накапливается», она анализируется через мета-индексы, что позволяет другим пользователям видеть не только рейтинг, но и статистический «профиль доверия».

Платформенный UI: Пользователи могут открыть встроенный UI-экран, чтобы увидеть свою «статистику доверия» и провести собственный анализ верификации других участников. Это создает петлю обратной связи: честные пользователи получают высокий рейтинг, а попытки манипуляций становятся видимыми для всего сообщества.

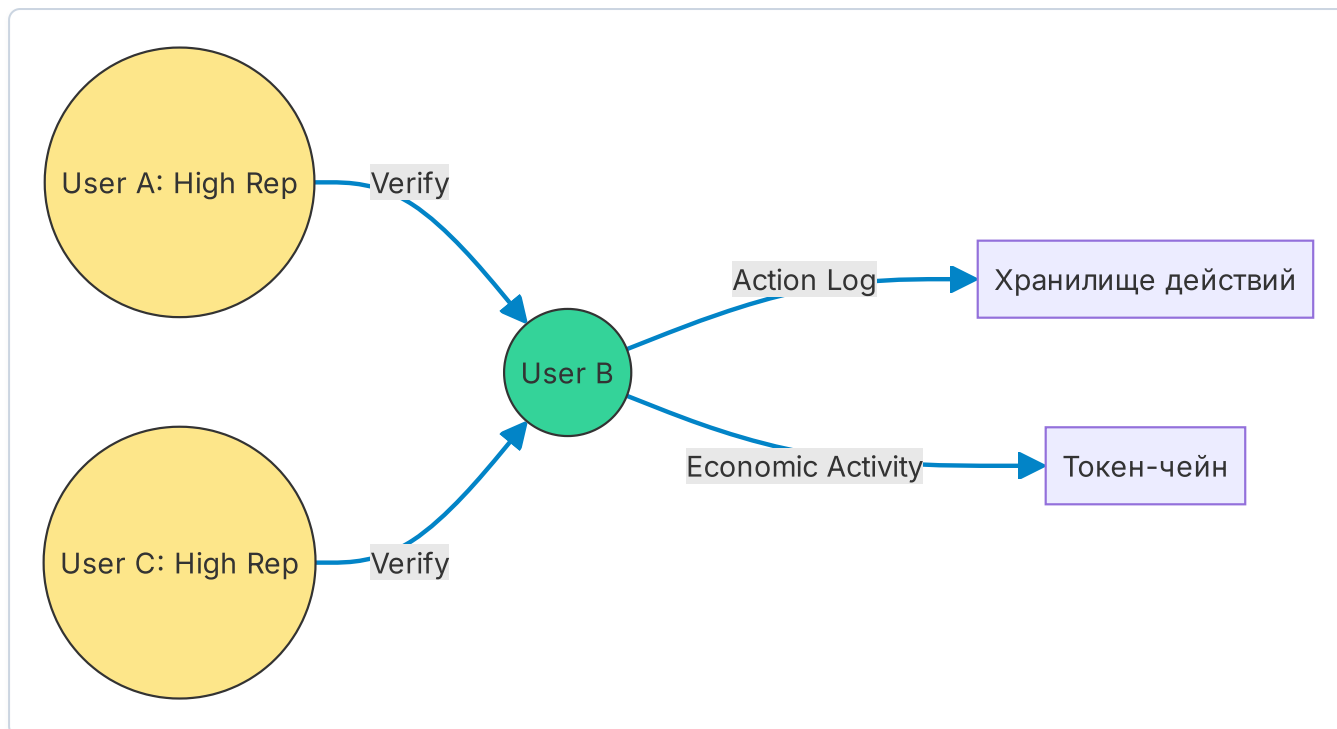


15.3. Репутационный граф и экономическое доверие

Репутация в сети «Забота» не является статичным показателем. Она подкрепляется:

Взаимным подтверждением: Высокорейтинговые пользователи подтверждают действия других, что повышает общий вес сети доверия.

Экономической активностью: Прозрачность транзакций через защищенный токен-чейн платформы (с сохранением конфиденциальности деталей контрактов) позволяет другим участникам верифицировать серьезность намерений партнера по сделке.



Интерфейсная прозрачность: Использование «платформенных экранов» (Platform Screens), которые гарантируют, что данные получены из доверенного хранилища, а не подменены злонамеренным UI.

16. КубГолос

16.1. КубГолос как инструмент государственного управления

Для государства «КубГолос» представляет собой критический актив. Это механизм «обратной связи» с населением, работающий в реальном времени, который позволяет:

Мониторинг общественного мнения: Государство получает возможность анализировать инициативные опросы граждан, которые часто являются более точными и менее предвзятыми, чем официальные замеры.

Масштабируемая демократия: Возможность проводить опросы любого уровня — от локальных административных единиц до общенациональных, обеспечивая высокую точность данных благодаря верификации участников через уровни регистрации (Блок 15).

Экспертиза по требованию: Возможность быстрого сбора мнений от профессиональных сообществ для принятия взвешенных управленческих решений.

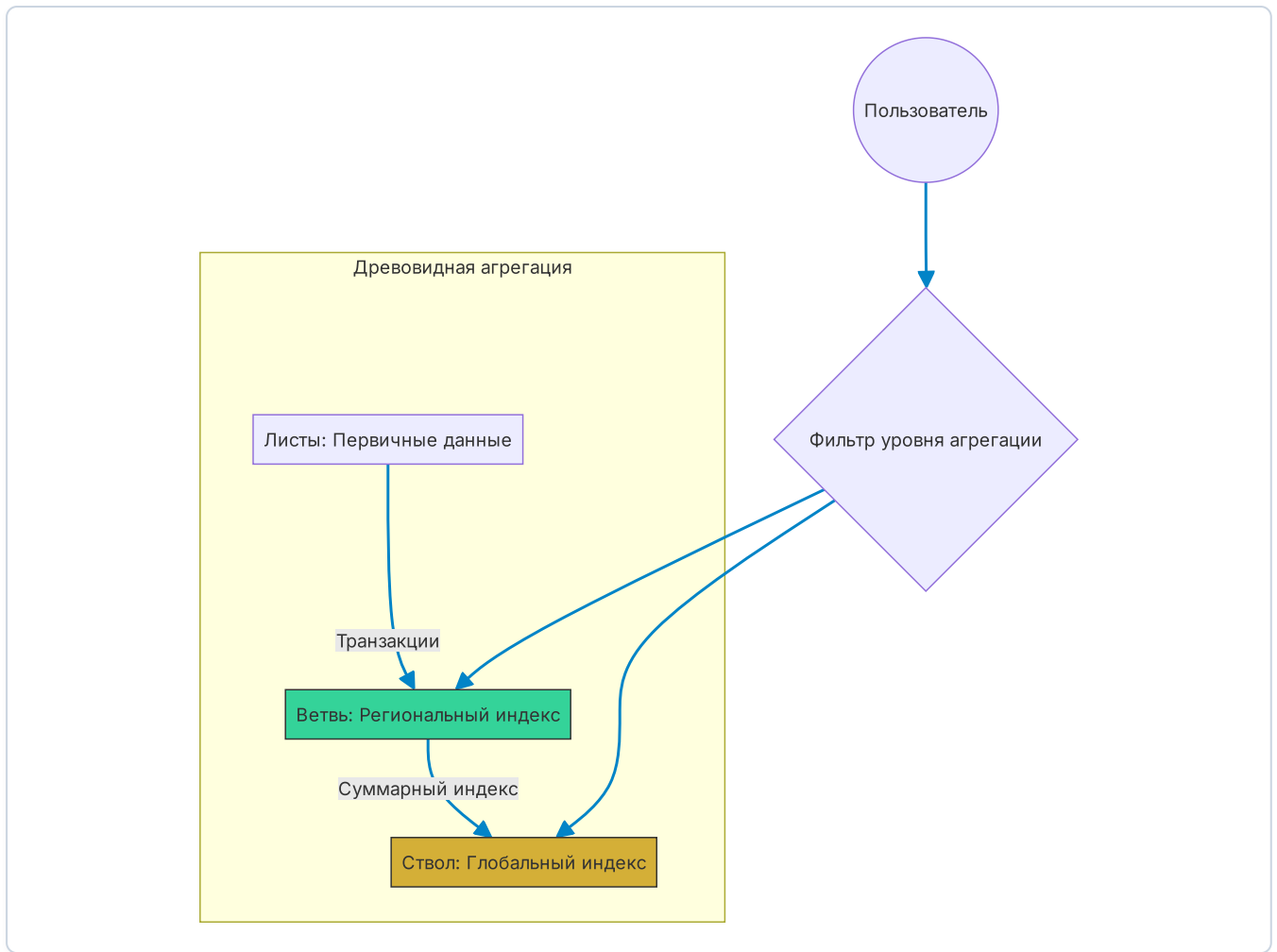
16.2. Иерархическая топология и производительность

Система индексации использует древовидную топологию (Tree-Topology), где каждый узел (Ветвь) агрегирует данные локально. Это обеспечивает:

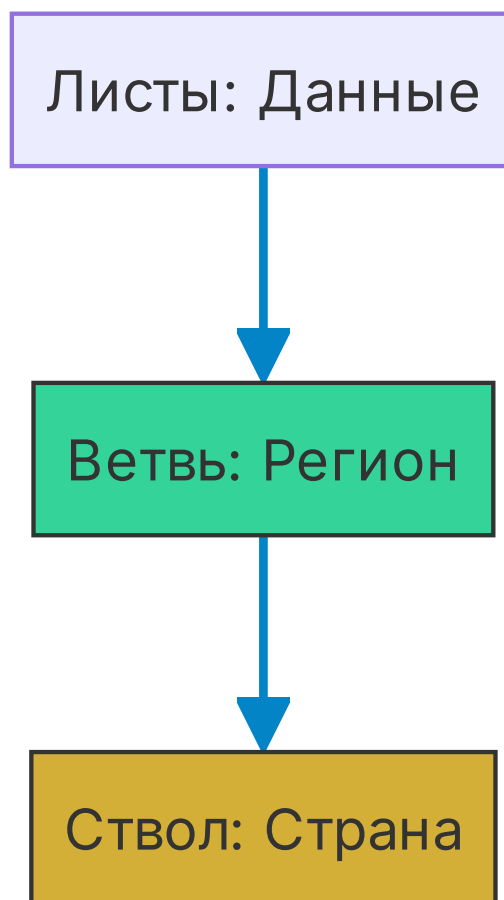
Высокую скорость: Агрегация происходит в реальном времени на каждом уровне иерархии.

Эффективность (DDoS-устойчивость): Доступ к опросу лимитирован участниками конкретной Ветви, что предотвращает внешние атаки.

Гибкость: Пользователь или администратор может запрашивать отчеты на любом уровне детализации: локальный район, регион или страна.



Иерархическая топология позволяет собирать экспертные мнения и общественную статистику в реальном времени, превращая систему в глобальную «нервную систему».



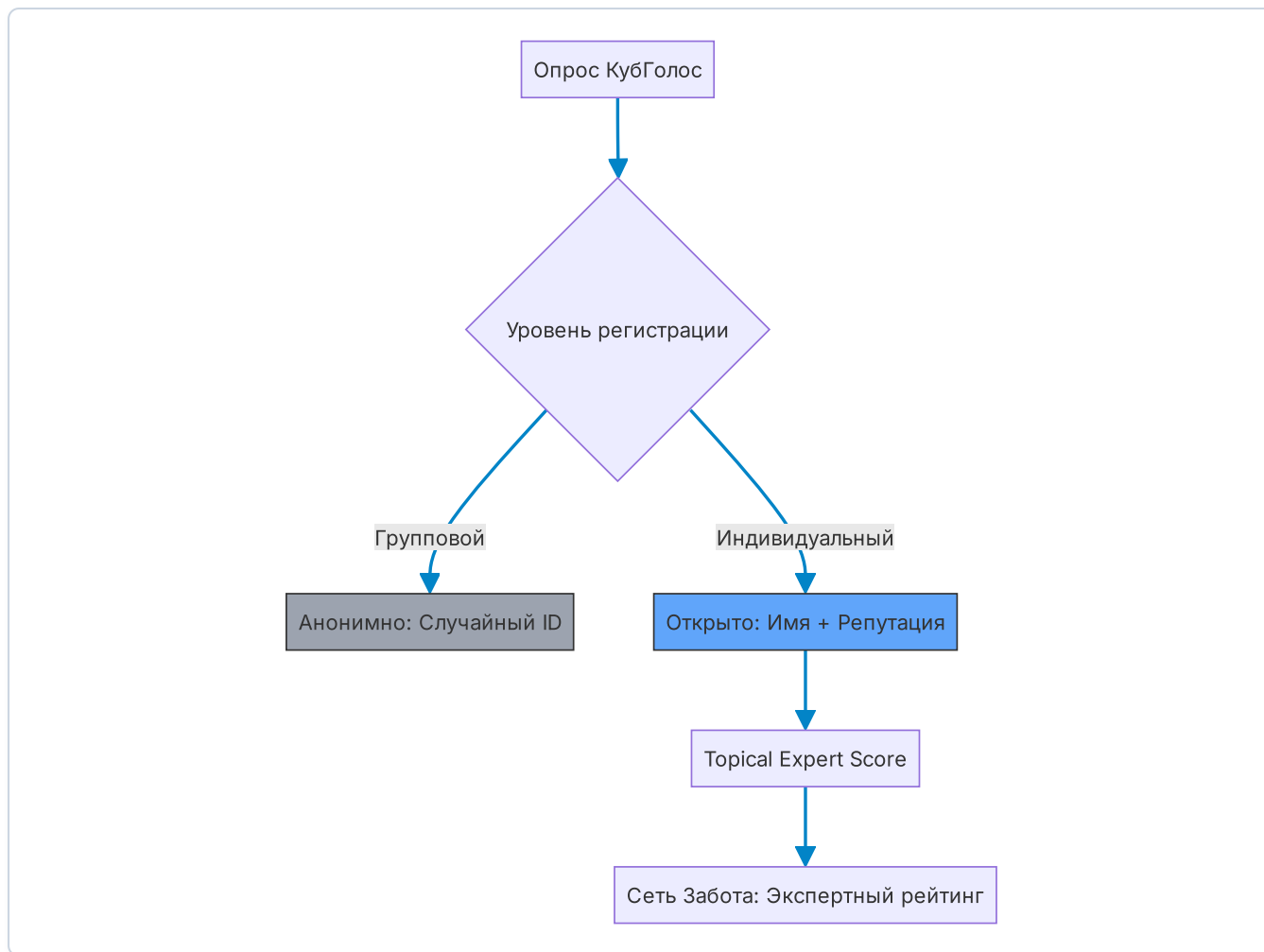
16.3. Модель верификации и саморегулирования

В отличие от полностью анонимных систем, «КубГолос» опирается на прозрачность регистрации.

Ответственность: Так как каждый голос подкреплен верифицированным статусом пользователя в Ветви, данные обладают высокой достоверностью.

Выбор участия: Пользователь сам решает, участвовать ли в опросе анонимно (через групповую регистрацию) или открыто (от своего имени, помещая имя в общественный контур), исходя из личного уровня комфорта и требуемого доверия.

Аналитическая ценность: Ветви предоставляют статистику не только по количеству голосов, но и по «качеству» выборки (репутация участников, их экспертность в теме), что дает правительству возможность лучше понимать структуру общественного мнения.



16.4. Достоверность через верификацию и экспертизу

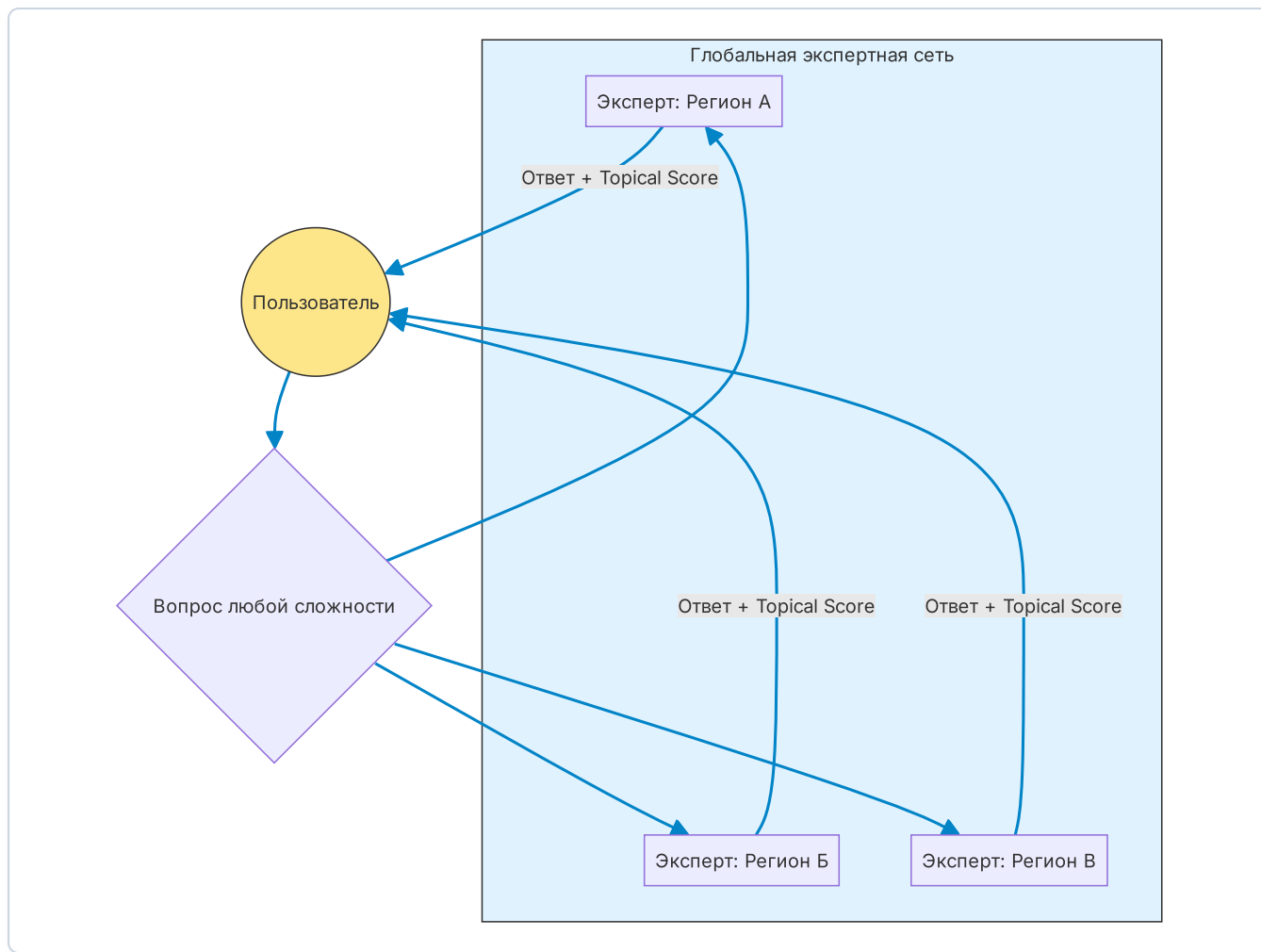
Данные в «КубГолос» обладают высокой ценностью благодаря интеграции с репутационной системой «Забота»:

Topical Expert Score: Ответы экспертов оцениваются не «в целом», а по конкретной теме. Пользователь видит не просто мнение, а уровень компетенции отвечающего.

Прозрачность: Система предоставляет пользователям инструменты для анализа уровня агрегации данных, позволяя самостоятельно определять, является ли мнение региональным или отражает глобальный экспертный консенсус.

16.6. КубГолос как «Золотая мечта» глобального знания

Помимо административной функции, система реализует модель глобального интеллектуального обмена. Это «We-система», где любой пользователь — от ребенка до ученого — может задать вопрос и получить верифицированные ответы от экспертов со всего мира в режиме реального времени, зная регион эксперта и принимая это во внимание.



Заключение

Использование ПК и телефонов пользователей как серверов обрабатывающих хранилища на совместных схемах данных, а так же использование логической древовидной структуры веток и соответствующей индексации информации на базе современных облачных оптимизаций позволяет:

- Обеспечить совместный доступ к данным
- Централизовать обработку данных
- Оптимизировать вычислительные мощности
- Обеспечить суверенитет данных государства в международной платформе
- Владеть своими данными
- Добровольно идентифицироваться, локально или лично
- Создавать приложения совместной обработки данных
- Обеспечить логическую изоляцию данных
- Обмениваться данными напрямую
- Прикреплять сообщества к местности